

基于微信小程序的 AviAI 鸟类智能识别系统

课程项目报告

何周屹（2312190419）

邱志翔（2312190432）

项目仓库：AviAI | 技术栈：微信小程序 + FastAPI + MySQL + Redis

报告日期：2026 年 6 月

一、项目介绍 [何周屹、邱志翔]

1.1 背景与问题陈述

观鸟正在从少数专业人士的爱好走向大众。越来越多的人在公园、湿地、校园里遇到不认识的鸟，想知道它叫什么、有什么习性，但传统的识别方式门槛很高：要么翻厚厚的图鉴一页页比对，要么在论坛发帖等别人回答，等到有结果时鸟早已飞走。对新手来说，仅凭“灰色的小鸟”这种模糊印象去检索，几乎无从下手。

我们做 AviAI，就是想把“看到鸟—认出鸟—记录鸟—分享鸟”这条链路压缩到一部手机上。用户掏出微信小程序，对准鸟拍一张照片，几秒钟内就能拿到物种名称、置信度和特征描述，识别结果会自动存进个人记录，还能一键生成文案发到社区和同好交流。

选择微信小程序作为载体，是因为它无需安装、即用即走，天然契合观鸟这种“突发、户外、轻量”的使用场景；用户不必为了认一只鸟专门下载一个 App。后端则把算力密集的图像识别和 AI 文案放在服务端，小程序只负责拍照、上传和展示，保证了前端的轻量化和跨设备一致性。

1.2 项目目标与核心功能

项目的总目标是交付一个可真实部署、端到端跑通的鸟类识别与社区平台。下面只列出当前已经实现并通过测试的功能，未完成的设想统一放在第十七章展望中。

1.2.1 功能性需求（已实现）

- ① 账号体系：邮箱 + 密码注册、登录、登出；JWT 鉴权；登出后 Token 立即失效。
- ② 鸟类识别：支持拍照或相册上传图片，服务端用 ResNet18 做本地分类、DeepSeek 视觉模型做细化识别，返回鸟名、置信度与图片地址，并写入识别记录。
- ③ 识别历史：按用户分页查询自己的历史识别记录。
- ④ 社区互动：发布图文动态、浏览时间线、查看详情、点赞、评论；动态支持关键词搜索。
- ⑤ AI 文案辅助：发帖前可让 AI 基于图片“生成”文案，或对已有文案“润色”，并返回 lite / enhanced 双版本。
- ⑥ 个人中心：查看与编辑个人资料。

1.2.2 非功能性需求（已实现）

- ① 性能：识别记录与社区列表均采用分页；后端提供进程内指标与平均响应时间统计。

- ② 可用性：提供 /health 与 /api/v1/health 健康检查，接入 Render 健康检查与 UptimeRobot 外部告警。
- ③ 安全性：密码使用 bcrypt 哈希、参数化查询防注入、敏感配置走环境变量、CI 内置 gitleaks 密钥扫描。
- ④ 可观测性：结构化 JSON 日志、Prometheus 文本指标、可选 Sentry 错误追踪。

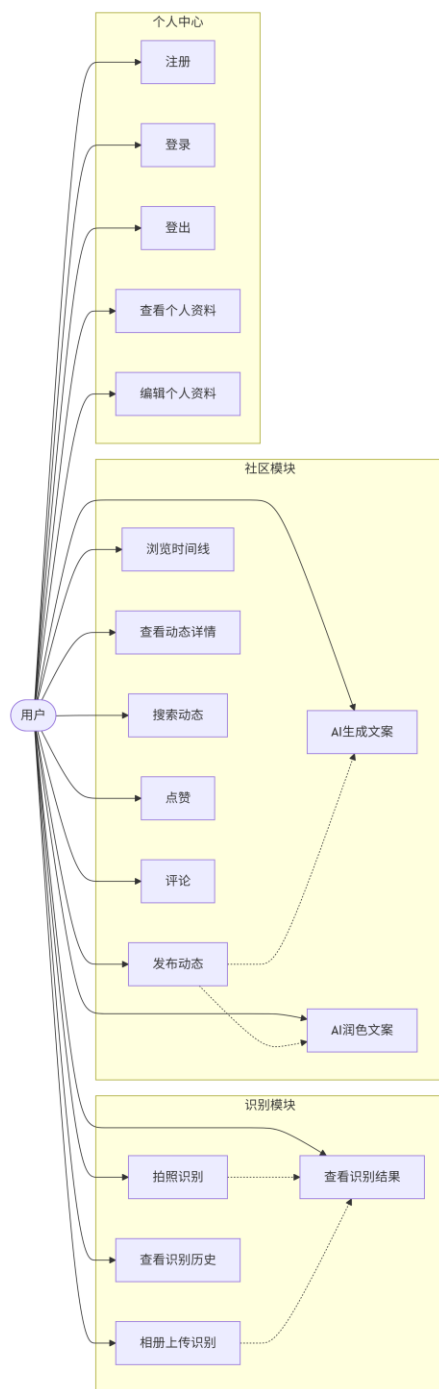


图 1-1 项目功能思维导图或用例图

1.3 技术选型

技术选型遵循一个原则：优先选团队熟悉、生态成熟、能免费部署的方案，把精力留给业务本身。

层次	技术选型	选择理由
前端	原生微信小程序 (WXML/WXSS/JS)	无需安装、贴合户外轻量场景；原生开发避免编译层带来的体积与兼容问题。
后端	FastAPI (Python 3.11)	异步高性能、自带 OpenAPI 文档、Pydantic 校验，开发效率高。
数据库	MySQL 9.7	关系型数据天然适配用户/帖子/评论的强关联结构，事务可靠。
缓存	Redis 7	缓存社区时间线等热点读，降低数据库压力。
AI 能力	PyTorch ResNet18 + DeepSeek 视觉/文本 API	本地模型保证基本可用与零调用成本，云端大模型提升识别细化与文案质量。
鉴权	自实现 HS256 JWT	无额外依赖，逻辑透明可控，便于实现 Token 撤销。
部署	Docker + Render	一份 Dockerfile 多处运行；Render 连 GitHub 推送即自动部署并提供 HTTPS。

1.4 团队分工

本项目为两人团队，按“后端 + 工程化”与“前端 + 设计”两条主线分工，文档与测试共同承担。

成员	学号	主要负责模块
何周屹	2312190419	后端实现、API 设计、软件架构、AI 工程化、安全设计、CI/CD、Docker、监控
邱志翔	2312190432	前端实现、UI/UX 设计、云服务部署、前端测试与安全

说明：项目介绍、软件测试、功能展示、总结展望等章节由两人共同完成。

二、版本控制与团队协作 [何周屹、邱志翔]

2.1 分支策略

项目采用基于 GitHub Flow 改良的三层分支模型，兼顾个人开发隔离与主干稳定。

1. master: 生产主干，始终保持可部署状态，只接受来自 develop 的合并，受分支保护规则约束。

2. develop: 集成分支，汇聚各功能分支，用于联调与预发布验证。

3. feature/<姓名>-<主题>: 个人功能分支，例如 feature/HeZhouyi-backend、feature/QiuZhixiang-cloud，一个任务一条分支，完成后通过 PR 合并。

保护规则：master 与 develop 禁止直接推送，必须经 Pull Request；合并前要求 CI 全部通过。

Default

Branch	Updated	Check status	Behind	Ahead	Pull request
master	last week	✓ 6 / 6		Default	

Your branches

Branch	Updated	Check status	Behind	Ahead	Pull request
develop	last week	✓ 12 / 12	4	0	#89
feature/Hezhouyi-monitor	3 weeks ago	✓ 8 / 8	7	0	#86
feature/Hezhouyi-docker	last month	✓ 4 / 4	26	0	#74
feature/Hezhouyi-backend-safe	last month	✓ 4 / 4	50	0	#73
Hezhouyi-backend-ci	last month		62	1	

View more branches >

Active branches

Branch	Updated	Check status	Behind	Ahead	Pull request
develop	last week	✓ 12 / 12	4	0	#89
dependabot/npm_and_yarn/frontend/eslint-10.5.0	2 weeks ago	✗ 5 / 6	3	1	#88
dependabot/npm_and_yarn/frontend/prettier-3.8.4	2 weeks ago	✗ 5 / 6	3	1	#87
feature/Hezhouyi-monitor	3 weeks ago	✓ 8 / 8	7	0	#86

图 2-1 GitHub 仓库 Branches 页面

2.2 提交规范

提交信息遵循 Conventional Commits 约定，格式为 type(scope): subject，常用 type 包括 feat、fix、docs、refactor、test、chore、ci。例如仓库中真实的提交：feat(monitors): add Prometheus metrics endpoint、fix(ci): stabilize health metrics tests。

PR 流程：开发者在 feature 分支完成后发起 PR，填写改动摘要、测试情况与关联说明；CI 触发后端测试、前端测试、Docker 构建与安全扫描；全部通过且经审查后方可合并。

2.3 协作统计

截至报告撰写时，项目主仓库累计合并 67 个 Pull Request，提交总量约 190 余次，贡献按成员的 GitHub 账号分布如下（数据来自 git shortlog）。

成员 / 账号	提交次数	主要贡献方向
hzy2005	154	后端、AI、架构、CI/CD、安全、监控
JoinMike333	37	前端、云部署相关提交
dependabot[bot]	10	依赖安全自动升级

关键 PR 示例：#74 Docker 化、#73 后端安全加固、#78 云部署、#79 监控接入，均带有改动说明并经过 CI 校验后合并。项目使用 GitHub Issues / PR 进行任务跟踪与代码审查。

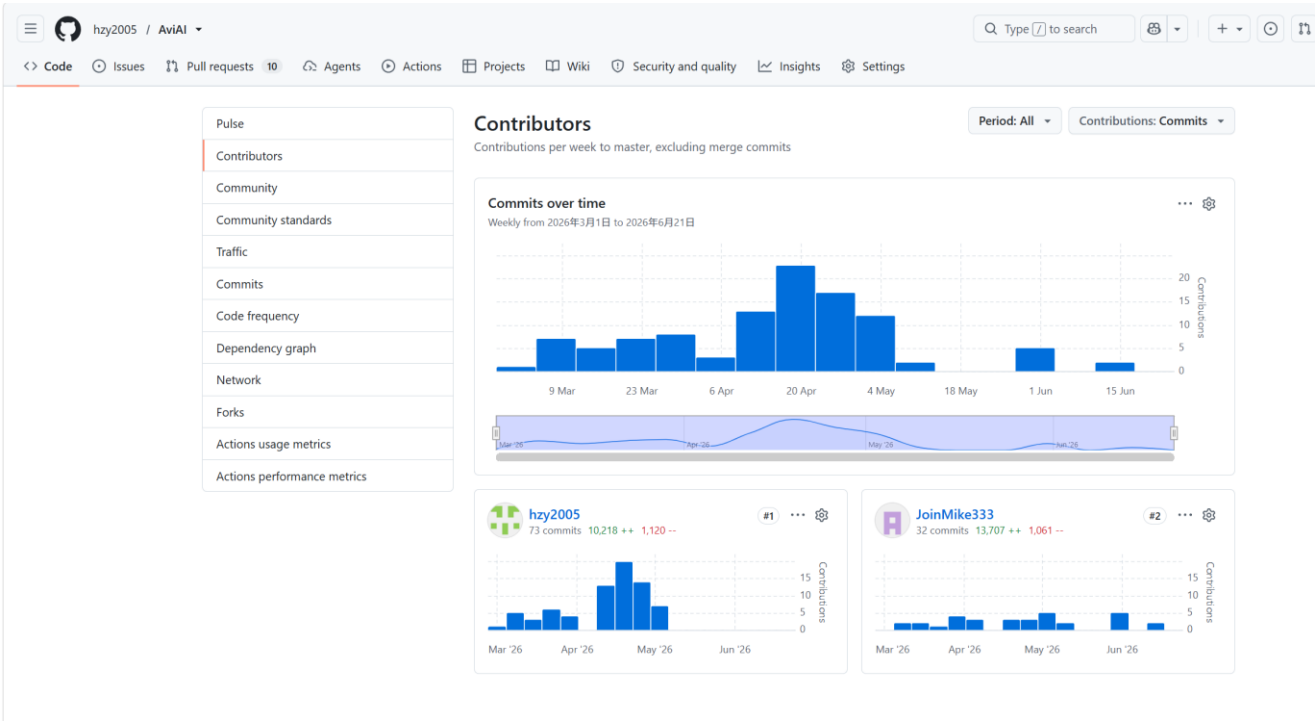


图 2-2 各成员提交量与代码行数展示

二、版本控制与团队协作 [何周屹、邱志翔]

3.1 用户画像与场景分析

AviAI 面向两类典型用户。

① **观鸟新手 / 自然爱好者**：周末在公园、湿地遇到不认识的鸟，希望立刻知道它是什么。他们对专业术语不敏感，最看重“拍一下就出结果”的即时反馈和好看易懂的结果展示。

② **进阶观鸟者 / 学生**：有一定知识基础，关注识别置信度、物种特征与保护等级，并乐于在社区记录和分享自己的观测，形成个人观鸟档案。

典型场景：用户在户外发现目标鸟 → 打开小程序进入首页 → 拍照或从相册选图 → 等待识别 → 查看结果卡片（鸟名、置信度、特征）→ 跳转百科了解更多，或一键生成文案发到社区。整个动线围绕“快”和“少打断”设计，核心操作不超过三步。

3.2 界面原型设计

小程序共 12 个页面，按功能聚成五个模块：账号（splash / login / register）、识别（index 首页 / recognize / bird-detail）、社区（community / community-detail）、百科（encyclopedia）、个人中心（profile / profile-edit / history）。页面跳转遵循设计说明中的交互约定：登录后进入首页，底部 Tab 在首页、社区、百科、个人中心间切换，点击识别结果或社区/百科中的鸟图进入对应详情页。

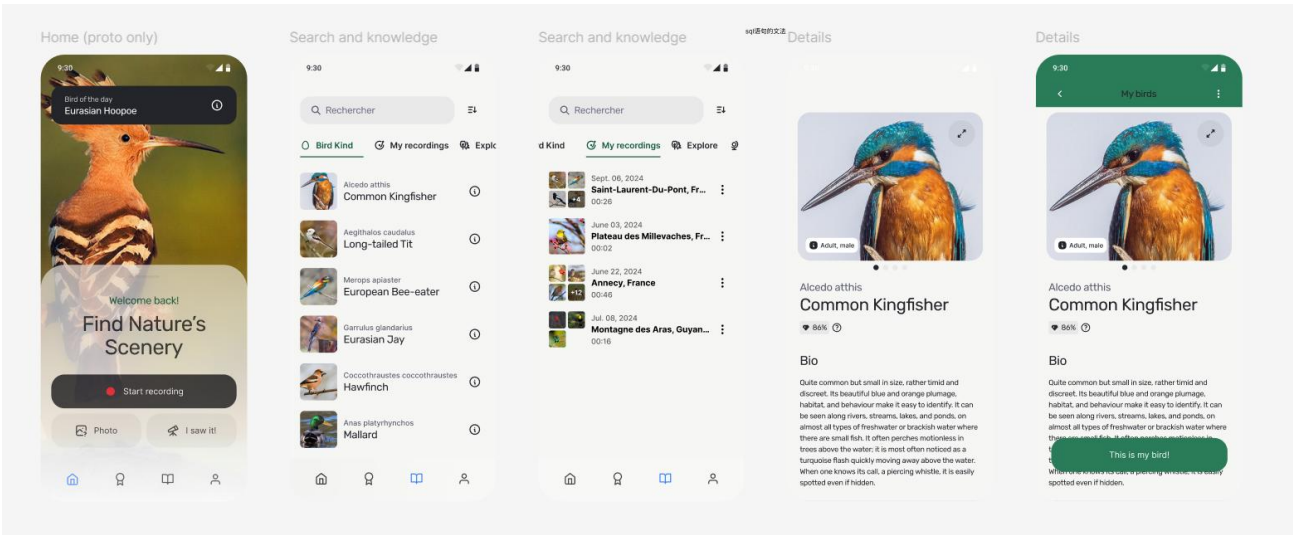


图 3-1 识别模块原型图

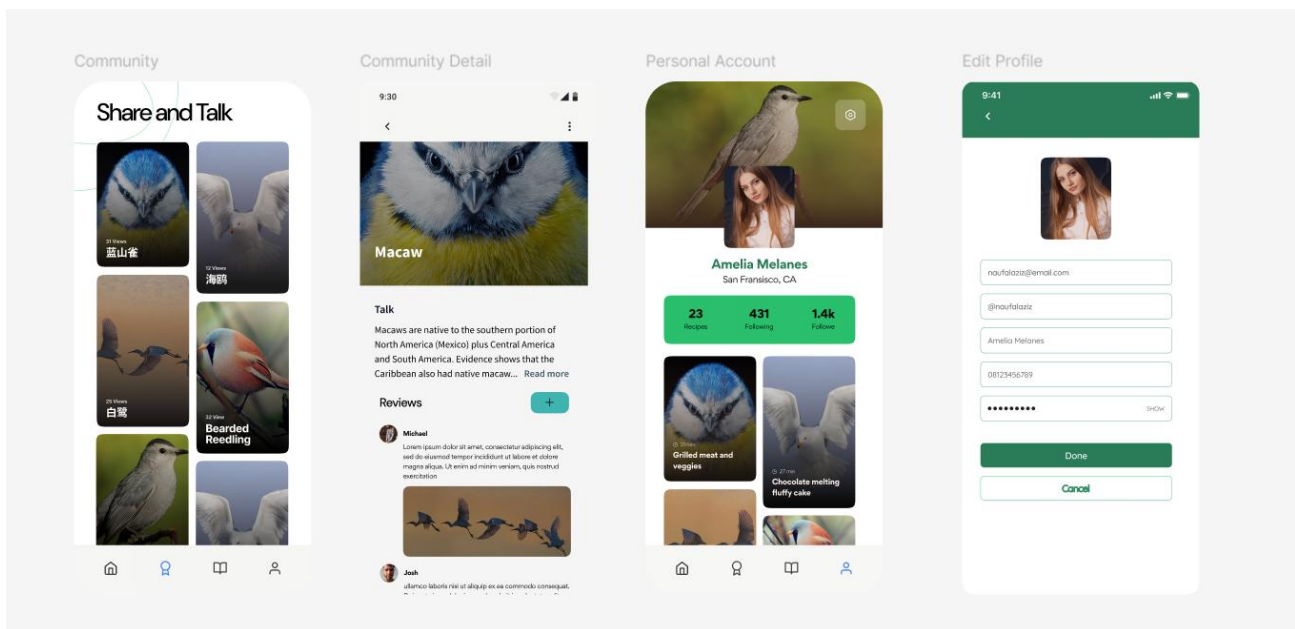


图3-2 社区与个人中心模块原型图

设计理念：界面以白色为底、绿色（#297C57 / #34B663）为主色调，呼应自然与鸟类主题，视觉干净、对比清晰。优点是风格统一、辨识度高、对新手友好；缺点是绿色系在弱光环境下层次较弱，后续可补充深色模式。实现难度上，识别结果卡片与社区图文流的多图排版是相对复杂的部分。

3.2.1 交互设计原则

拍照优先：首页把相机/上传入口放在最显眼的位置，减少用户思考成本。

屏幕适配：使用 rpx 弹性单位适配不同尺寸机型，保证关键按钮在小屏上也易于点击。

即时反馈：上传、识别、点赞等异步操作均有 loading 与结果提示，避免用户重复点击。

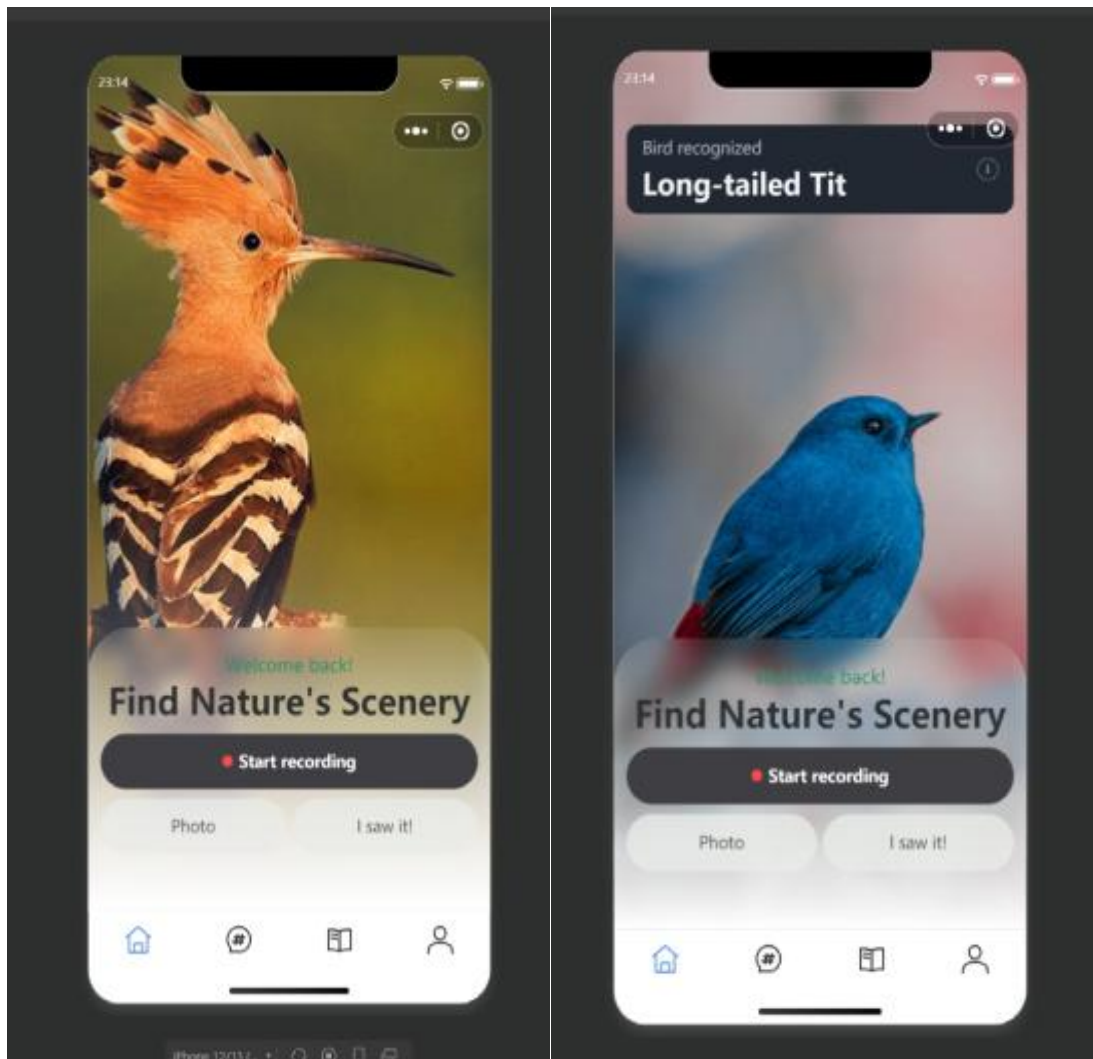
防误操作：删除动态、退出登录等不可逆操作前给出确认提示。

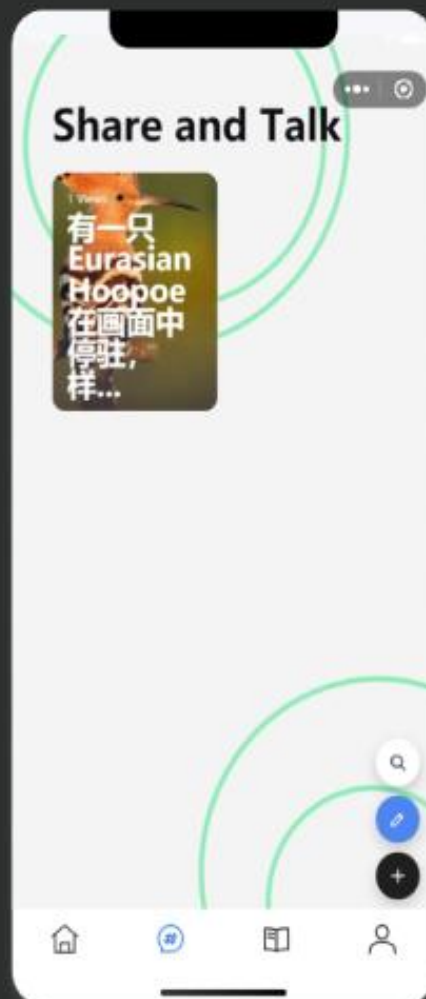
3.2.2 用户体验设计

加载速度：图片上传与识别分离，先快速反馈“上传成功”，再等待识别结果，减少白屏等待感。

操作便捷：社区点赞、评论采用局部刷新，无需整页重载。

视觉一致：统一的卡片、按钮、空状态与加载组件，降低用户的认知负担。





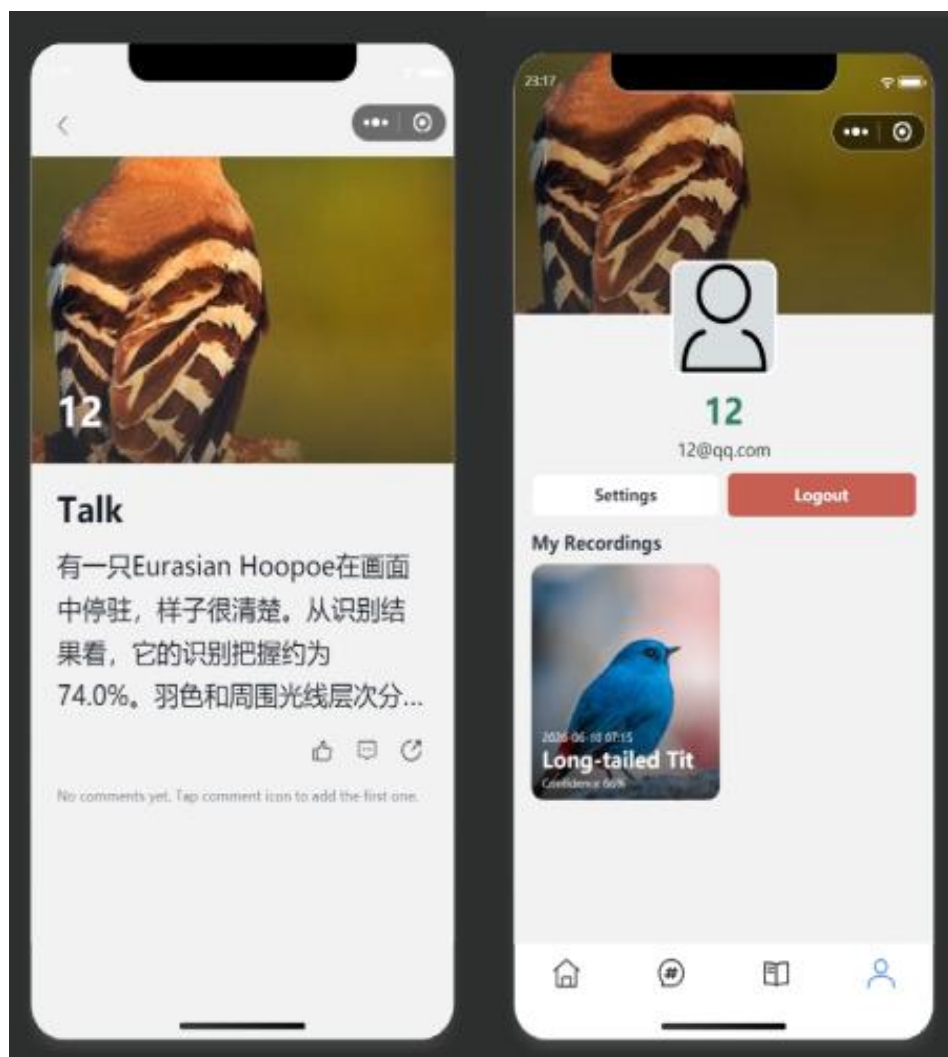


图3-4 关键页面真机运行截图

四、软件架构设计 [何周屹]

4.1 整体架构

AviAI 采用“微信小程序前端 + FastAPI 业务服务 + 数据基础设施”的分层架构。前端负责交互与展示；业务服务负责接口、鉴权与业务编排，并在内部完成鸟类识别（PyTorch 本地模型 + DeepSeek 视觉 API）；MySQL 存结构化数据，Redis 缓存热点读，本地 uploads 目录（生产可换对象存储）保存图片文件。

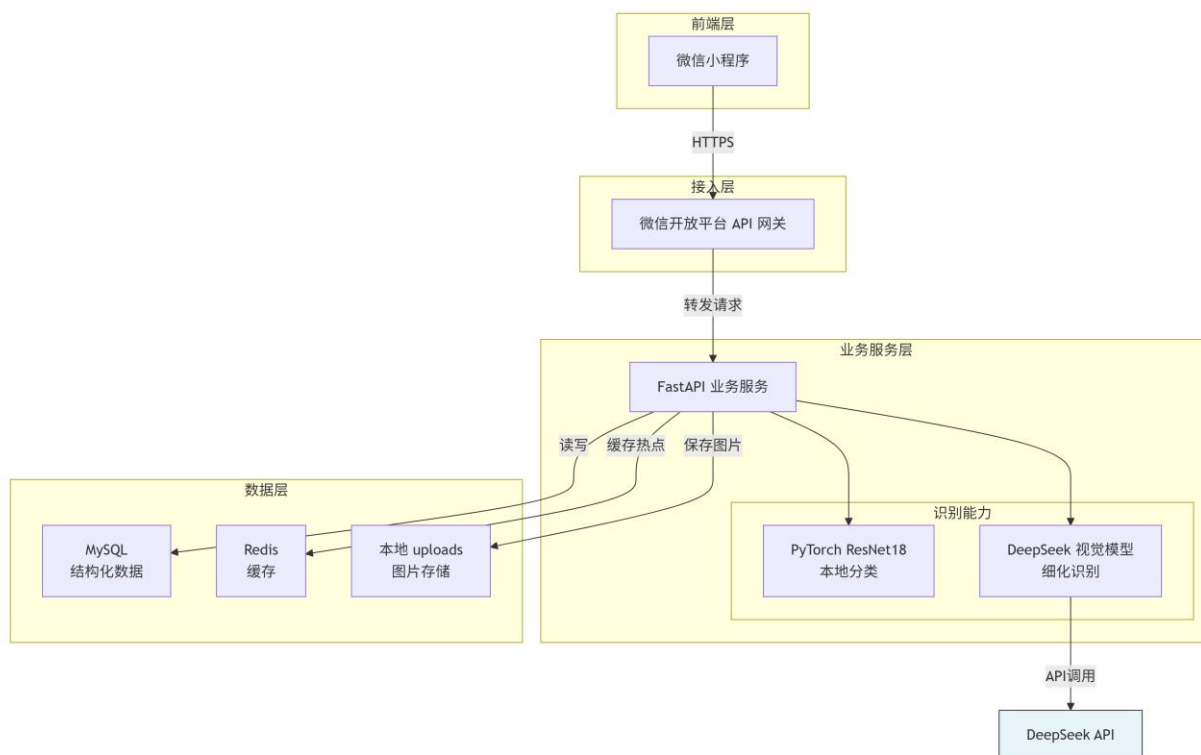


图 4-1 系统总体架构图

这样分层职责清晰、耦合低：前端不感知模型细节，模型与业务可分别迭代。但缺点是单体业务服务在高并发下需要水平扩展，且本地文件存储不利于多实例共享，这两点已在展望中规划为后续优化方向。

4.2 技术架构分层

4.2.1 表现层（前端）

前端为原生小程序，`app.js/app.json` 管理全局配置，业务页面在 `pages/`，网络与工具能力在 `utils/`。所有 HTTP 请求统一经过 `utils/request.js` 封装，集中处理 Token 注入、错误提示与离线 Mock 路由；`config/env.js` 负责切换开发 / 局域网真机 / 生产三套环境地址。这样组织保证了请求逻辑只有一处，便于统一维护。

4.2.2 业务逻辑层（后端）

后端按“路由层 → 业务层 → 数据访问层”组织。**路由层**（`app/routes/`，含 `auth`、`users`、`birds`、`posts`、`deps`）只做参数接收与鉴权守卫；**核心业务**集中在 `app/services/api_service.py`，包括识别、AI 文案、社区互动等逻辑；**数据访问**通过 SQLAlchemy ORM 完成。约定路由不写业务、业务不直接碰请求对象，避免逻辑重复。

4.2.3 数据访问层

使用 SQLAlchemy ORM 映射 users、bird_records、posts、post_images、comments、likes 等表，数据库会话在 app/db/session.py 统一管理。高频读（如社区时间线）通过 Redis 缓存，分页查询统一返回 list/total/page/pageSize 结构。表结构与索引详见第七章。

4.3 关键设计决策

决策点	选择	理由
接口风格	REST 而非 GraphQL	接口数量适中、关系清晰，REST 更直观、调试简单，FastAPI 还能自动生成 OpenAPI 文档。
数据库	MySQL 而非 MongoDB	用户—帖子—评论—点赞是强关系数据，需要事务与外键约束，关系型更合适。
鉴权	自实现 HS256 JWT	无状态、易水平扩展；自实现便于加入 Token 撤销列表实现可靠登出。
识别方案	本地 ResNet18 + 云端 DeepSeek	本地模型零成本兜底保证可用，云端大模型提升细化识别与描述质量。

五、API 设计 [何周屹]

5.1 设计原则

API 遵循 RESTful 风格，统一以 /api/v1 为前缀，使用名词复数表示资源、HTTP 动词表示操作。所有响应采用统一信封结构，成功为 {code:0, message:"ok", data:{...}}，失败时 data 为 null 并返回业务错误码。时间统一用 ISO 8601 格式。

错误码规范如下，业务码与 HTTP 状态码配合使用：

错误码	含义	典型场景
0	成功	正常返回
1001	参数错误	请求体校验失败（HTTP 400）
1002	未登录或 Token 无效	缺少或非法 Token（HTTP 401）
1003	无权限	越权访问他人资源（HTTP 403）
1004	资源不存在	访问不存在的帖子等（HTTP 404）

1005	服务内部错误	未预期异常（HTTP 500）
1006	上传文件不合法	图片类型/大小不符（HTTP 400）
1009	资源冲突	重复注册、重复点赞（HTTP 409）

5.2 接口文档

完整契约以仓库 docs/api.yaml（OpenAPI 3.0）为准，下面按模块给出接口总览。

5.2.1 用户认证接口

方法	路径	鉴权	说明
POST	/auth/register	否	用户注册
POST	/auth/login	否	登录，返回 token 与用户信息
POST	/auth/logout	是	登出，服务端撤销当前 Token
GET	/users/me	是	获取当前用户资料

5.2.2 识别相关接口

方法	路径	鉴权	说明
POST	/birds/recognize	是	multipart 上传图片（字段名 image），返回识别结果
GET	/birds/records	是	分页查询识别历史

5.2.3 社区与 AI 文案接口

方法	路径	鉴权	说明
POST	/posts	是	发布动态
GET	/posts	否	动态分页列表，支持 keyword 搜索
GET	/posts/{postId}	否	动态详情
PUT/DELETE	/posts/{postId}	是	更新/删除动态（仅作者）
POST	/posts/{postId}/like	是	点赞（重复点赞返回 1009）
POST	/posts/{postId}/comments	是	评论
POST	/posts/upload-image	是	先上传图片，返回 /uploads 地址

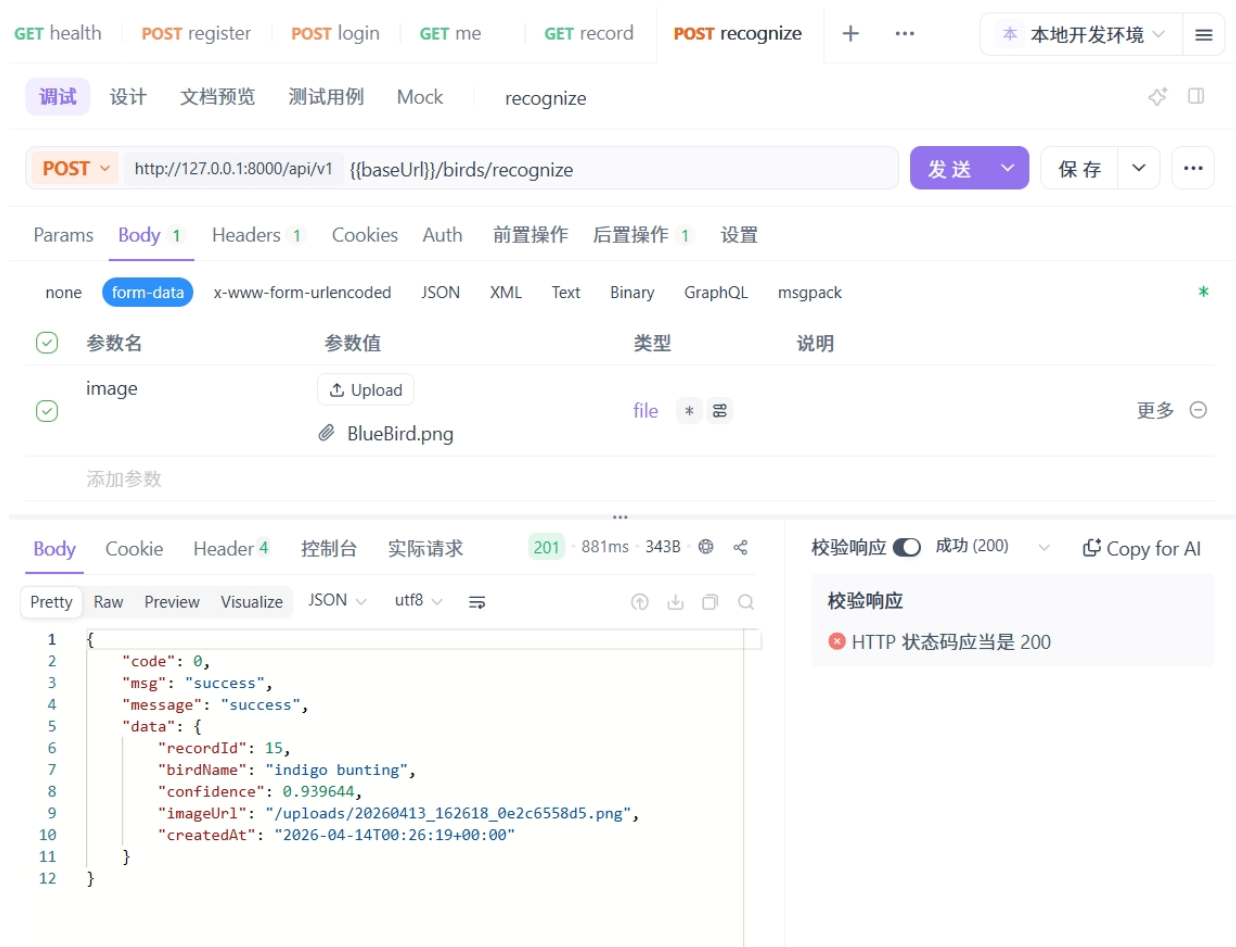


图 5-1 鸟类识别接口 /birds/recognize 调用结果

5.3 接口安全设计

鉴权采用 HS256 签名的 JWT，通过 `Authorization: Bearer <token>` 传递；服务端在依赖（`app/routes/deps.py`）中统一解析并校验 Token，登出时把 Token 加入撤销列表使其立即失效。权限上，鸟类记录、帖子的更新/删除等接口都会校验资源归属的 `user_id`，防止越权。敏感数据方面：密码用 `bcrypt` 哈希存储、传输走 HTTPS、DeepSeek 等密钥只从环境变量读取。

5.4 接口测试

后端用 `pytest + httpx` 对接口做契约测试，覆盖正常返回、鉴权失败、参数错误、越权与冲突等路径，其中 `test_api_contract.py` 专门固化前端关心的响应字段。CI 在 Python 3.11 / 3.12 两个版本上以 SQLite 内存库运行全部测试并统计覆盖率，详见第十章。

# file	line %	branch %	funcs %	uncovered lines
# config				
# env.js	100.00	50.00	100.00	
# pages				
# community				
# community.js	97.50	72.54	98.00	52 344-345 350-351 421 462-466 494-495
# login				
# login.js	81.82	83.33	50.00	16 20 24-26 45-47 58-61
# recognize				
# recognize.js	78.69	33.33	66.67	14-16 29-33 38-39 42-43 56
# register				
# register.js	80.23	64.29	60.00	21 25-27 35-37 40-42 45-47 78-81
# src				
# __tests__				
# api-mock.test.js	99.09	94.12	100.00	34-35
# coverage-boost.test.js	100.00	98.72	88.10	
# helpers				
# test-utils.js	98.41	89.29	100.00	96-97
# page-interactions.test.js	99.40	100.00	83.87	222-223
# api				
# auth.js	66.67	66.67	60.00	3-9 31-32 44-54
# birds.js	71.70	55.56	80.00	10-12 20-30 37
# client.js	66.67	100.00	0.00	3-5
# index.js	100.00	100.00	100.00	
# posts.js	97.03	75.00	100.00	34-35 40
# users.js	54.55	100.00	0.00	3-7
# utils				
# auth.js	100.00	100.00	100.00	
# format.js	80.00	44.44	100.00	5-6 10-11 23-24 30-31 39-40
# mock-api.js	92.27	66.45	100.00	109 126 135 142-143 148-149 161-162 263-264 325-326
329-330 343-344 357-358 389-390	418-419 426-427 430-431 457-458 463-464 491-492 498-499 508-509 514-515 534-535 540-541 555-556			
# request.js	83.96	76.92	100.00	23-24 30-35 51-59
# validator.js	100.00	66.67	100.00	
# all files	94.56	76.60	90.29	

图 5-2 接口测试运行结果

六、前端实现 [邱志翔]

6.1 技术栈与开发环境

前端采用原生微信小程序开发，页面由 WXML（结构）、WXSS（样式）、JS（逻辑）三部分组成，在微信开发者工具中调试运行。没有引入额外的编译框架，好处是产物体积小、与小程序运行时贴合、不受框架版本升级影响；代价是组件复用要靠自己封装。工程结构按职责划分：`pages/` 放 12 个业务页面，`utils/` 放请求、鉴权、Mock 等工具，`config/` 放环境配置，`static/` 放图标与设计资源。

一个关键的工程化设计是“请求统一出口”：所有接口调用都走 `utils/request.js`，它负责自动注入 `Authorization` 头、统一解析 `{code,message,data}` 信封、弹出错误提示，并在离线开发时把请求路由到 `utils/mock-api.js` 的本地假数据。`config/env.js` 通过开关在“本地模拟器 / 局域网真机 / 线上生产”三套 `baseURL` 间切换，真机联调时无需改业务代码。

6.2 核心功能模块实现

6.2.1 用户管理模块

登录注册页通过 `request.js` 调用 `/auth/login` 与 `/auth/register`，登录成功后由 `utils/auth.js` 把返回的 `JWT` 存入本地 `storage`，后续请求自动携带。退出登录时调用 `/auth/logout` 并清除本地 `Token`，保证服务端撤销与客户端清理同步。考虑到的因素主要是 `Token` 的安全存储与失效处理：本地只存 `Token` 不存密码，过期或被撤销时请求会收到 401，前端据此引导用户重新登录。

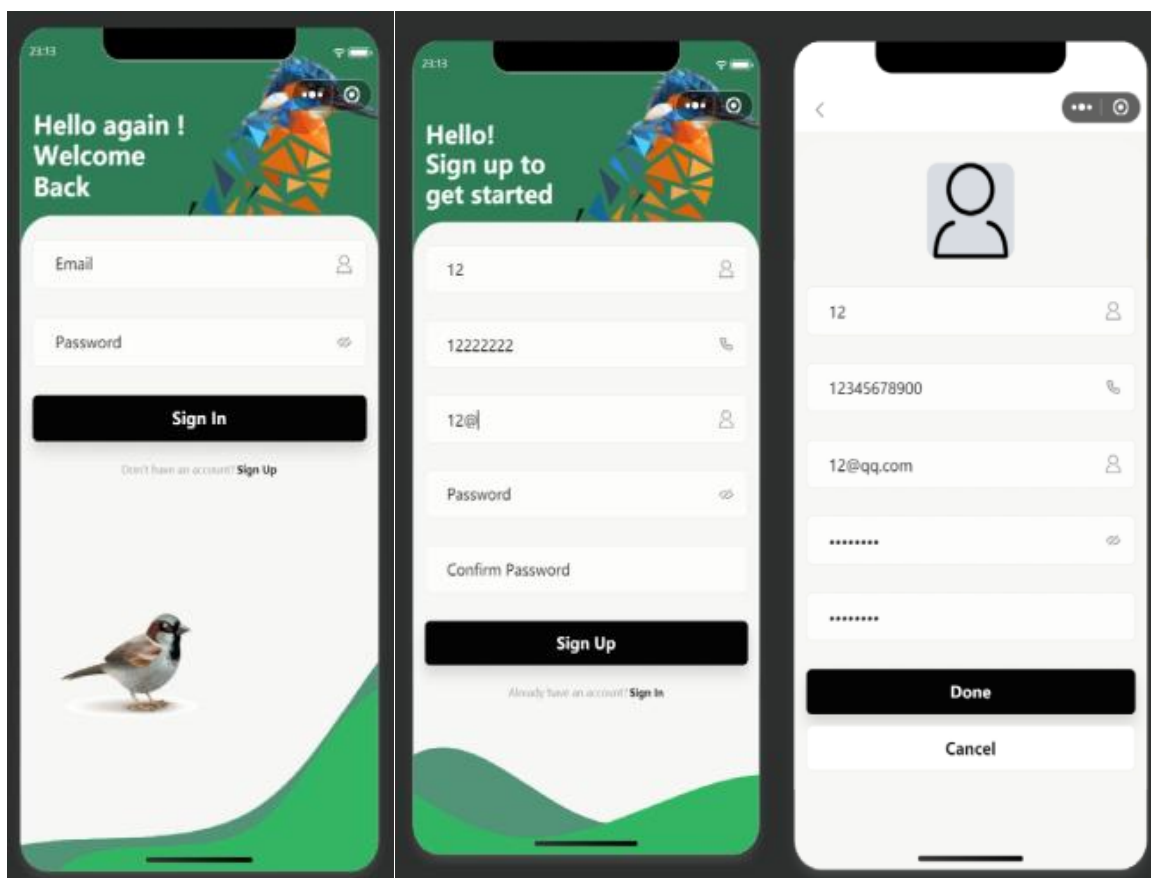


图 6-1 登录注册以及个人中心资料编辑界面

6.2.2 课堂点名 / 鸟类识别模块

识别是前端最核心的链路。用户在首页选择拍照或相册，拿到本地临时图片路径后，以 multipart/form-data（字段名 image）上传到 /birds/recognize，等待服务端返回鸟名、置信度与图片地址，再渲染成结果卡片。为降低等待焦虑，上传与识别在交互上分阶段反馈，并提供跳转百科详情与历史记录入口。

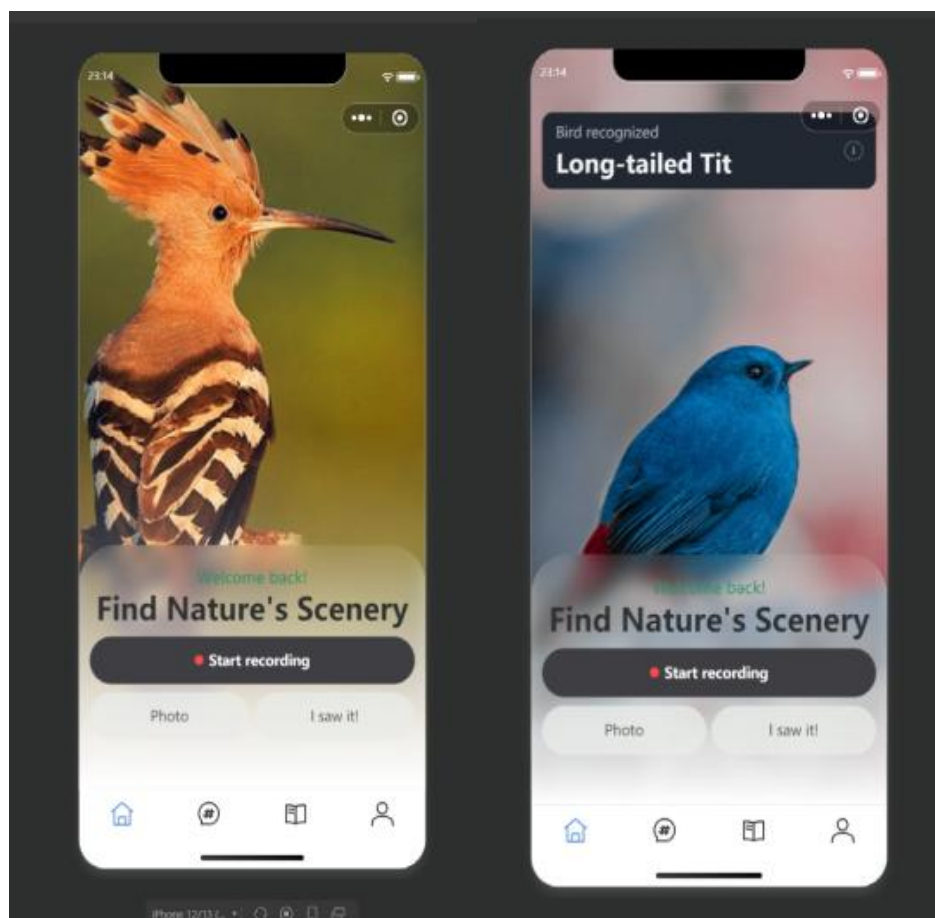


图 6-2 鸟类识别

6.2.3 数据展示模块

社区时间线与识别历史都是典型的分页列表。前端采用上拉触底加载下一页，配合后端 `list/total/page/pageSize` 分页结构，避免一次性拉取全部数据。采用的技术是小程序原生 `scroll-view` + 触底事件 + 本地列表拼接；优点是实现简单、内存可控，缺点是超长列表的 DOM 节点会累积，后续可引入虚拟列表优化。实现难点在于点赞/评论后的局部状态同步——通过只更新对应帖子对象的计数字段而非整页重载来解决。

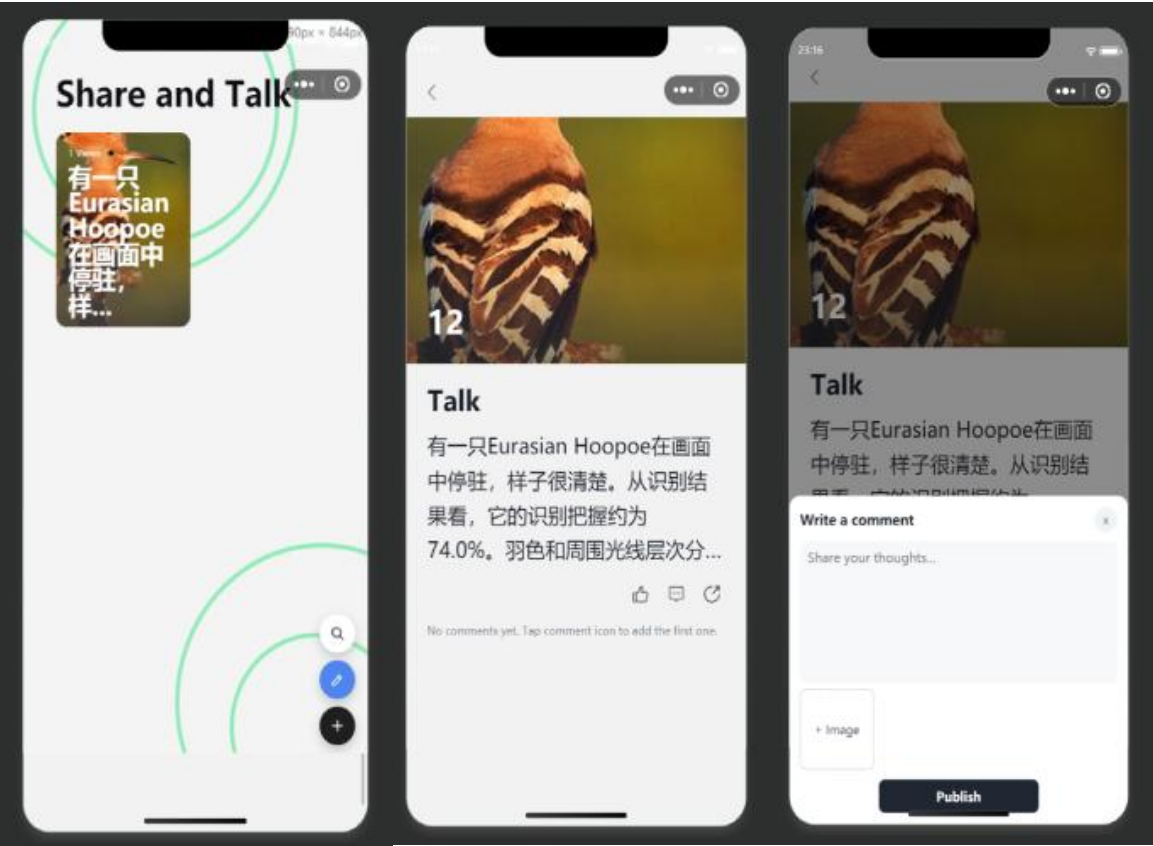


图 6-3 社区列表页与帖子详情页

6.3 性能优化实践

前端在以下几个方向做了优化，因小程序场景与 Web 不同，部分指标以体验改善为主。

- ① 图片处理：识别上传前对图片做尺寸压缩，减少上传耗时与流量。
- ② 分页与触底加载：长列表分页拉取，避免首屏一次性渲染过多节点。
- ③ 局部刷新：点赞/评论只更新单条数据，避免整页重渲染。
- ④ 请求收敛：统一请求封装内做错误兜底与必要的节流，减少重复请求。

优化项	优化前	优化后	说明
首页列表首屏	一次性加载全部	分页 + 触底加载	首屏请求数据量显著下降，滚动更流畅
识别图片上传	原图直传	压缩后上传	上传耗时与流量下降（请补充真机实测数据）
点赞交互	整页刷新	局部更新计数	点赞响应即时，无明显闪烁

注：上表 Before/After 的具体数值建议用微信开发者工具的 Network / Performance 面板实测后补全。

6.4 兼容性处理

使用 rpx 弹性单位适配不同屏幕宽度，关键交互在小屏机型上保证可点击区域；对低版本基础库可能缺失的 API 做能力判断与降级处理；针对不同机型的相机/相册权限，在调用前检查授权状态并引导用户开启。

七、后端实现 [何周屹]

7.1 技术栈与架构

后端基于 FastAPI（应用版本 0.3.0）构建，使用 Uvicorn 作为 ASGI 服务器。选择 FastAPI 的原因是它把请求校验（Pydantic）、依赖注入、自动 OpenAPI 文档和异步支持整合在一起，开发效率高且类型安全。应用在 app/main.py 中完成初始化：注册 CORS 中间件、挂载 /uploads 静态目录、注册四个业务路由，并统一注册校验异常与全局异常处理器，把异常映射为统一错误码。

```
23 app = FastAPI(title="AviAI API", version="0.3.0")
24 configure_logging()
25 configure_sentry()
26 logger = get_logger(__name__)
27 APP_VERSION = "0.3.0"
28
29 BACKEND_ROOT = Path(__file__).resolve().parents[1]
30 UPLOADS_DIR = BACKEND_ROOT / "uploads"
31 UPLOADS_DIR.mkdir(parents=True, exist_ok=True)
32
33 app.add_middleware(
34     CORSMiddleware,
35     allow_origins=["*"],
36     allow_credentials=True,
37     allow_methods=["*"],
38     allow_headers=["*"],
39 )
40
41 app.mount("/uploads", StaticFiles(directory=str(UPLOADS_DIR)), name="uploads")

159 app.include_router(auth_router)
160 app.include_router(users_router)
161 app.include_router(birds_router)
162 app.include_router(posts_router)
163
```

7.2 核心业务模块实现

7.2.1 用户认证与授权

认证逻辑集中在 app/core/auth.py。密码用 bcrypt 加盐哈希；登录时兼容历史 SHA-256 哈希并在验证成功后自动重哈希升级。JWT 为自实现的 HS256：手工构造 header.payload.signature 三段，payload 含 sub（用户 ID）、exp（过期时间）、jti（唯一 ID）。登出时把 Token 写入带过期清理的撤销列表，decode 时先查撤销列表，实现“登出即失效”。

7.2.2 课程/识别业务逻辑

识别入口 `recognize_bird_for_user` 先做鉴权与文件校验（扩展名白名单），保存图片到 `uploads`，再调用 `_predict_bird_from_image` 推理，最后把鸟名、置信度、图片地址写入 `bird_records` 表。推理策略是“本地模型优先、降级兜底”：优先用 `torchvision` 预训练 ResNet18 做分类（CPU 友好、零成本），若模型不可用则退回到基于图像平均色调的轻量识别，保证接口永远有结果返回。

```
259 def _predict_bird_from_image(file_path: Path) -> tuple[str, float]:
260     # Primary: Torch pretrained model (CPU-friendly, open-source).
261     # Fallback: lightweight color-based recognizer if model unavailable.
262     try:
263         return _predict_bird_from_torch(file_path)
264     except Exception:
265         pass
266
267     with Image.open(file_path) as image:
268         rgb_image = image.convert("RGB").resize((64, 64))
269         pixels = list(rgb_image.getdata())
270
271     total = len(pixels) or 1
272     avg_r = sum(p[0] for p in pixels) / total
273     avg_g = sum(p[1] for p in pixels) / total
274     avg_b = sum(p[2] for p in pixels) / total
275
276     if avg_b > avg_r + 18 and avg_b > avg_g + 12:
277         return "Common Kingfisher", 0.82
278     if avg_g > avg_r + 12 and avg_g > avg_b + 8:
279         return "Mallard", 0.78
280     if avg_r > avg_b + 15 and avg_r > avg_g + 8:
281         return "Eurasian Hoopoe", 0.74
282     if (avg_r + avg_g) / 2 > avg_b + 20:
283         return "European Bee-eater", 0.70
284     return "Long-tailed Tit", 0.66
```

7.2.3 社区业务逻辑

社区模块负责发帖、时间线读取、点赞与评论。发帖时帖子与多图（`post_images`）一并入库；点赞通过 `likes` 表的 (`user_id`, `post_id`) 唯一约束防重复，重复点赞返回 1009 冲突；评论支持 `parent_id` 嵌套回复。热点时间线可借助 Redis 缓存提升读性能。

7.2.4 数据统计分析

后端通过进程内指标收集器统计请求总数、错误数、错误率、平均响应时间与各状态码计数，经 `/api/v1/metrics` 与 `/metrics` 暴露，作为运营与性能观察的基础数据。

7.3 数据库设计

数据库使用 MySQL（CI 与本地测试用 SQLite 内存库），通过 Alembic 迁移脚本管理表结构演进，现有三个迁移版本分别建立用户表、识别记录表，以及帖子/评论/点赞相关表。核心表如下：

表名	作用	关键字段 / 约束
users	用户账号与资料	username/email 唯一，password_hash，建 idx_username/idx_email
bird_species	鸟类百科数据源	common_name/scientific_name，保护等级，detail_images(JSON)
recognition_records	识别历史	user_id/bird_species_id 外键，confidence，source_type，按 created_at 建索引
posts	社区动态	user_id 外键，content，status，按 user_id/created_at 建索引
post_images	帖子多图	post_id 外键 ON DELETE CASCADE，sort_order
comments	评论	post_id/user_id 外键，parent_id 支持嵌套回复
likes	点赞	UNIQUE(user_id, post_id) 防重复点赞

设计上用外键约束保证关联完整性，对高频查询字段（用户名、邮箱、外键、时间）建索引，敏感字段（密码）哈希存储，并为未来功能预留字段（如个人简介、观鸟年限）。

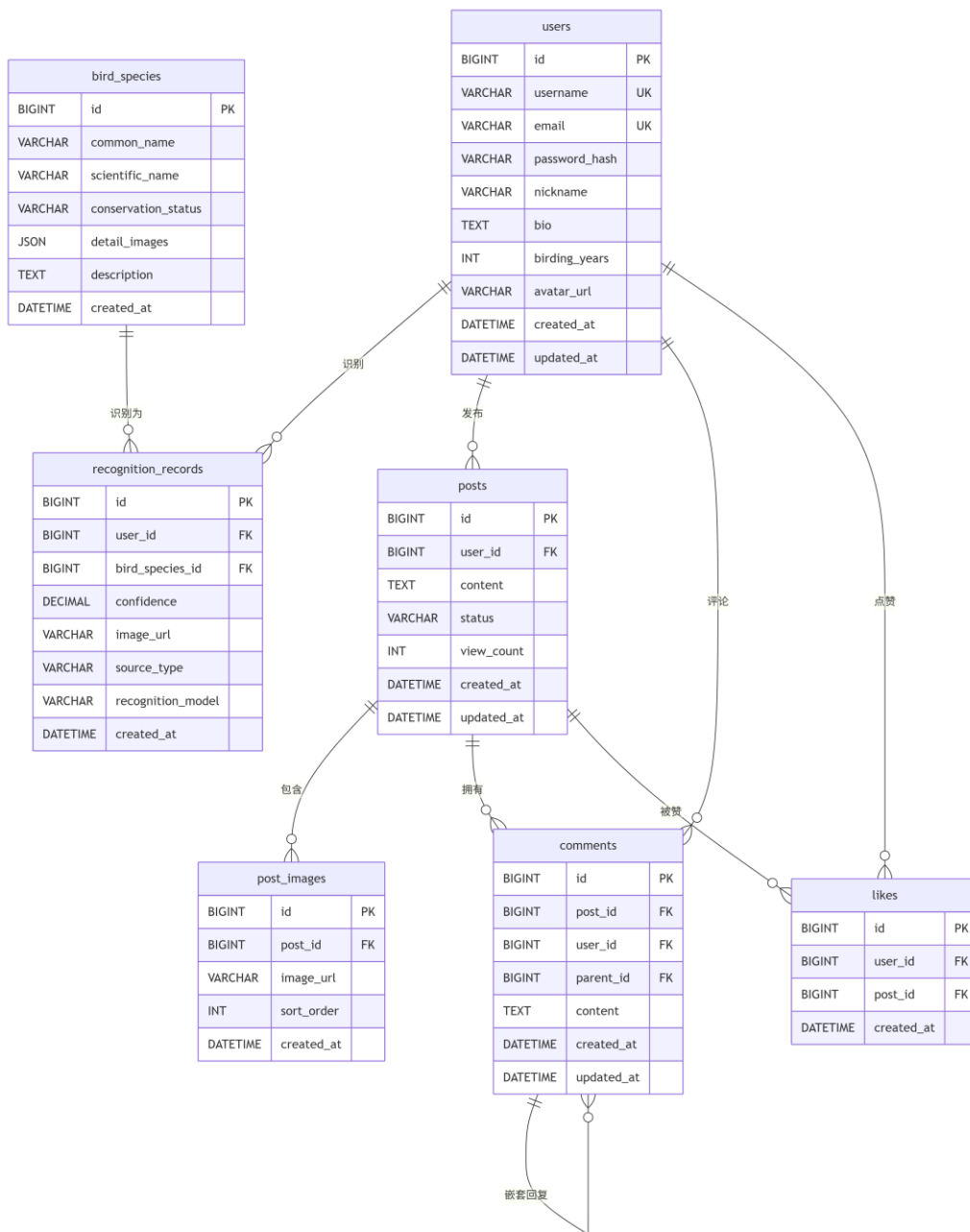


图 7-1 数据库 ER 图

7.4 中间件与工具集成

7.4.1 缓存机制

引入 Redis 7 缓存社区时间线等热点读数据，降低数据库压力；compose 中 Redis 作为独立服务编排。

7.4.2 日志系统

app/utils/logger.py 输出 JSON 结构化日志，并接管 uvicorn 访问日志使其统一为 JSON 格式，便于在 Docker 与 Render Logs 中检索。请求日志包含方法、路径、状态码、耗时、客户端地址等字段。

7.4.3 其他中间件

自定义 HTTP 中间件在每个请求前后记录指标与日志；全局异常处理器把 SQLAlchemy 异常、校验异常与 HTTP 状态码统一映射为业务错误码，避免向客户端泄露内部堆栈。

7.5 性能优化实践

- 查询分页：**识别记录与社区列表均分页返回，避免全表扫描与大结果集。
- 索引优化：**对外键与高频查询字段（created_at、username、email）建立索引。
- 模型懒加载：**ResNet18 模型按需加载并用锁保证单例，避免每次请求重复初始化。
- Redis 缓存：**热点时间线读走缓存，减少数据库往返。

优化项	优化前	优化后	说明
识别记录查询	全量返回	分页 + 索引	响应时间与内存占用下降（请补充实测）
模型加载	每次请求加载	进程内单例懒加载	首次后识别延迟显著降低
时间线读取	直查数据库	Redis 缓存	热点读 QPS 提升（请补充实测）

注：上表 Before/After 数值建议结合 /api/v1/metrics 的平均响应时间在压测前后取值填写。

八、AI 工程化应用 [何周屹]

8.1 AI 辅助开发实践

开发过程中团队把 AI 当作“结对编程伙伴”，在多个环节提效。

- ① 使用工具：ChatGPT、GitHub Copilot、Claude。
- ② 代码生成：用 Copilot 补全样板代码（Pydantic schema、路由骨架、测试用例脚手架）。
- ③ 代码审查：用 AI 从 OWASP Top 10 视角审查认证、鉴权、上传与数据库访问代码（见第九章）。
- ④ 文档编写：用 AI 生成 API 文档与章节初稿框架，再由人工核对与补充。
- ⑤ 经验总结：AI 擅长生成结构化样板与发现常见疏漏，但对项目特定约定容易出错，必须逐条核对与代码的一致性，不能直接采信。



图 8-1 与 AI 工具的开发对话

8.2 AI 辅助故障排查

在 CI 阶段曾出现健康检查与指标相关测试不稳定的问题（对应提交 fix(ci): stabilize health metrics and frontend URL tests）。排查时把失败的测试输出、相关接口实现与指标收集

逻辑作为上下文提供给 AI，AI 指出问题可能源于指标收集器在并发请求下的计数竞争与时间断言过严，建议固定测试中的请求顺序并放宽时间相关断言。据此调整后该问题得到解决：

1. 问题描述：CI 中健康指标 / 前端 URL 相关测试间歇性失败。
2. 提供给 AI 的上下文：失败用例输出、metrics 收集器实现、相关测试代码。
3. AI 的分析与建议：并发计数竞争 + 断言过严，建议稳定顺序、放宽时间断言。
4. 实际结果：调整后 CI 测试恢复稳定通过。

8.3 AI 功能集成

项目把 AI 能力真正集成进了业务，分两块。

其一是鸟类识别：本地 PyTorch ResNet18 做基础分类，云端 DeepSeek 视觉模型做细化识别与描述生成，模型与参数（DEEPSEEK_VISION_MODEL、温度、max_tokens、超时）均由环境变量配置。

其二是社区发帖 AI 文案（POST /posts/ai-copywriting），支持两种模式：generate 在用户只上传图片时自动生成分享文案；polish 在用户已写文案时进行润色，并返回 lite（轻润色）与 enhanced（增强润色）双版本。为保证质量与稳定性，服务端结合识别结果作为提示上下文、对输出做结构与长度约束、做质量检查（最小长度、重复词、模板化短语过滤）并自动重试一次，失败时回退到兜底文案保证可用；返回的 aiMeta（模型、耗时、重试次数、是否兜底等）用于排查“文案是否贴图”等效果问题。

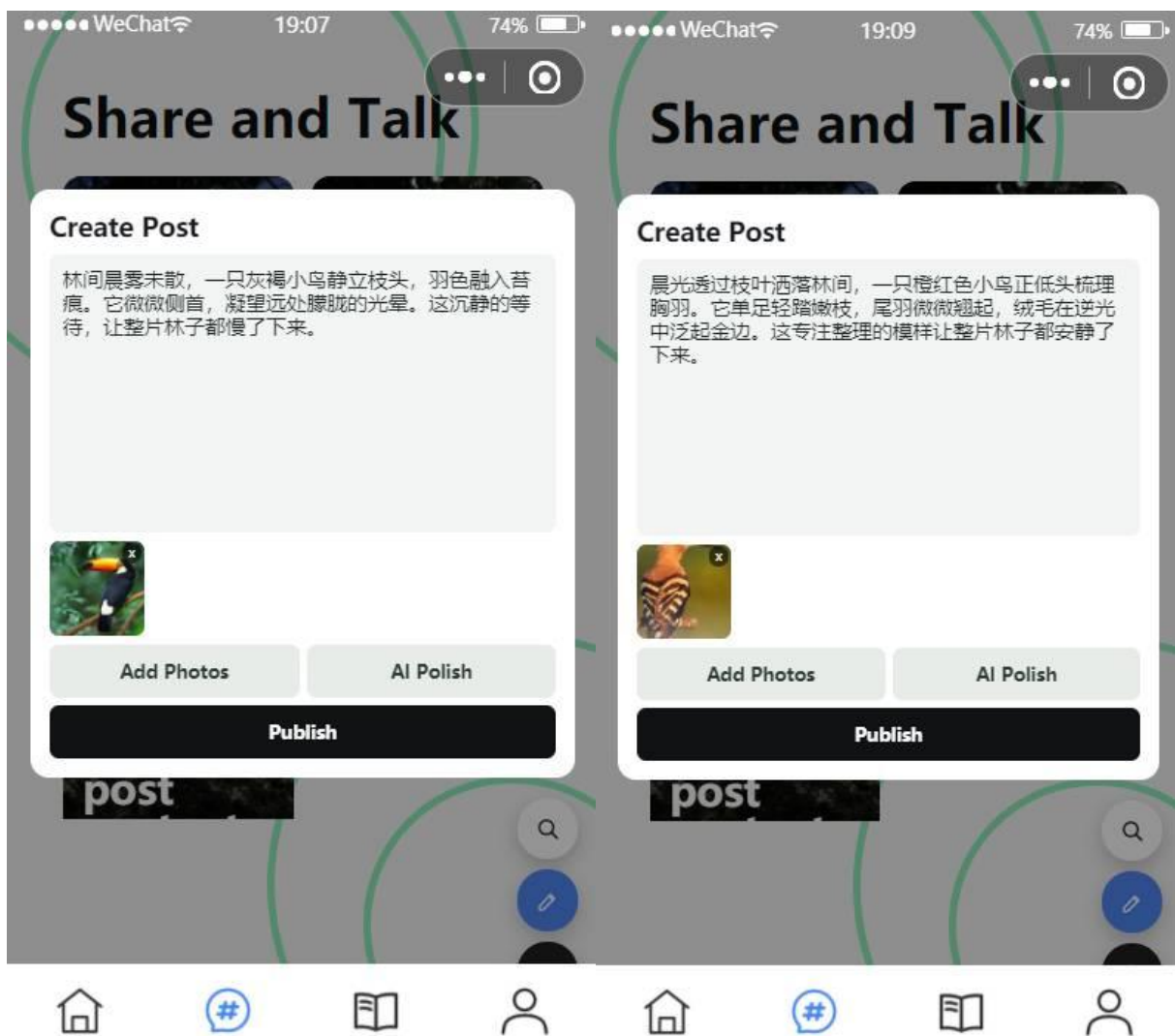


图8-2 AI 文案生成/润色的前端效果

九、安全设计 [何周屹]

9.1 安全威胁分析

结合 OWASP Top 10，对系统主要威胁做了梳理：SQL 注入、失效的访问控制（未鉴权接口、越权访问）、认证与凭据保护不足（弱密码哈希、Token 不失效）、敏感信息暴露（硬编码密钥）、以及使用含已知漏洞的依赖组件。团队据此用 AI 辅助做了一轮安全审查并完成加固。

9.2 安全防护措施

9.2.1 身份认证与授权

采用 HS256 JWT 鉴权，Token 含过期时间；登出后当前 Token 进入撤销列表立即失效。users/me、birds/*、posts 写接口、AI 文案接口均通过依赖注入获取当前用户；鸟类记录与帖子的更新/删除会校验资源所属 user_id，防止越权。

9.2.2 输入验证与 SQL 注入防护

所有请求体经 Pydantic 校验类型与格式；数据库访问统一走 SQLAlchemy ORM 与参数化查询，审查未发现字符串拼接 SQL，从根本上规避注入。文件上传限制扩展名白名单，拒绝非图片文件。

9.2.3 敏感数据保护

密码升级为 bcrypt 加盐哈希存储，并兼容旧 SHA-256 哈希的平滑迁移；传输使用 HTTPS；DeepSeek API Key、数据库口令、SECRET_KEY 等一律通过环境变量注入，默认配置仅保留占位值，.env 已在 .gitignore 中忽略。

9.2.4 其他安全措施

统一错误处理避免泄露内部堆栈；CORS 在生产可收敛白名单；后续可补充速率限制与安全 HTTP 头。

9.3 安全审计

团队完成了“AI 辅助审查 + 至少两处漏洞修复 + 安全检查清单 + CI 自动化扫描”的闭环。已修复项包括：密码哈希升级为 bcrypt、logout 实现 Token 失效、清理默认敏感配置、升级含已知漏洞的依赖（fastapi 0.115.5→0.136.1、python-multipart 0.0.12→0.0.27、Pillow 10.4.0→12.2.0、SQLAlchemy 2.0.36→2.0.49 等）。

依赖扫描结果：前端 npm audit fix 后 found 0 vulnerabilities；后端 pip-audit 初扫发现 11 个漏洞，升级后复扫 No known vulnerabilities found。CI 中通过 gitleaks-action 自动扫描提交历史中的潜在密钥泄露（.github/workflows/security.yml）。





All checks have passed

8 successful checks





 CI / backend (3.11) (pull_request)

Successful in 1m







 CI / backend (3.11) (push)

Successful in 1m





 CI / backend (3.12) (pull_request)

Successful in 1m





 CI / backend (3.12) (push)

Successful in 1m





 CI / frontend (pull_request)

Successful in 10s





 CI / frontend (push)

Successful in 11s





 Security Scan / gitleaks (pull_request)

Successful in 10s





 Security Scan / gitleaks (push)

Successful in 7s





No conflicts with base branch

Merging can be performed automatically.

Merge pull request



You can also merge this with the command line. [View command line instructions.](#)

Still in progress? [Convert to draft](#)

图 9-1 CI 安全扫描通过截图

十、软件测试 [何周屹、邱志翔]

10.1 测试策略

测试分两层推进：后端用 `pytest + httpx` 做单元与接口契约测试，前端用 Node 内置 `test runner` 做页面交互与 API Mock 测试。目标是核心业务路径（认证、识别、社区）有用例覆盖，并以测试固化前后端契约。所有测试纳入 CI，作为合并的质量门禁。

10.2 单元测试

后端测试分布在 `tests/` 下：`test_services.py`（业务逻辑，51 个）、`test_api.py`（接口，19 个）、`test_api_contract.py`（契约，13 个）、`test_core_modules.py`（核心模块如鉴权，18 个）、`test_ai_copywriting_regression.py`（AI 文案回归，3 个），合计约 104 个用例；整体行覆盖率约 84.1%（`coverage.xml`）。前端测试约 47 个，覆盖页面交互与请求 Mock。

10.3 集成测试

接口测试以 `httpx` 直接驱动 FastAPI 应用，覆盖注册→登录→鉴权→识别→发帖→点赞评论的主链路，并验证错误码（401 未登录、403 越权、404 不存在、409 重复点赞）。CI 在 Python 3.11 与 3.12 两个版本、SQLite 内存库下运行，保证跨版本一致。

10.4 端到端测试

当前以接口契约测试 + 前端交互测试覆盖端到端关键路径；尚未引入独立 E2E 框架，已列入后续计划（如结合微信小程序自动化测试工具）。

10.5 测试结果汇总

测试类型	用例数	通过率	覆盖率
后端单元/服务测试	约 88	100%	与后端整体覆盖率合并统计
后端接口/契约测试	约 32	100%	与后端整体覆盖率合并统计
后端整体测试	约 104	100%	行覆盖约 84.1%
前端交互/Mock 测试	约 47	100%	见 lcov 报告

注：用例数为按文件统计的近似值，最终数字以 CI 测试报告为准；建议附 CI 测试与覆盖率截图。

# file	line %	branch %	funcs %	uncovered lines
# config				
# env.js	100.00	50.00	100.00	
# pages				
# community				
# community.js	97.50	72.54	98.00	52 344-345 350-351 421 462-466 494-495
# login				
# login.js	81.82	83.33	50.00	16 20 24-26 45-47 58-61
# recognize				
# recognize.js	78.69	33.33	66.67	14-16 29-33 38-39 42-43 56
# register				
# register.js	80.23	64.29	60.00	21 25-27 35-37 40-42 45-47 78-81
# src				
# __tests__				
# api-mock.test.js	99.09	94.12	100.00	34-35
# coverage-boost.test.js	100.00	98.72	88.10	
# helpers				
# test-utils.js	98.41	89.29	100.00	96-97
# page-interactions.test.js	99.40	100.00	83.87	222-223
# api				
# auth.js	66.67	66.67	60.00	3-9 31-32 44-54
# birds.js	71.70	55.56	80.00	10-12 20-30 37
# client.js	66.67	100.00	0.00	3-5
# index.js	100.00	100.00	100.00	
# posts.js	97.03	75.00	100.00	34-35 40
# users.js	54.55	100.00	0.00	3-7
# utils				
# auth.js	100.00	100.00	100.00	
# format.js	80.00	44.44	100.00	5-6 10-11 23-24 30-31 39-40
# mock-api.js	92.27	66.45	100.00	109 126 135 142-143 148-149 161-162 263-264 325-326 329-330 343-344 357-358 389-390 418-419 426-427 430-431 457-458 463-464 491-492 498-499 508-509 514-515 534-535 540-541 555-556
# request.js	83.96	76.92	100.00	23-24 30-35 51-59
# validator.js	100.00	66.67	100.00	
# all files	94.56	76.60	90.29	

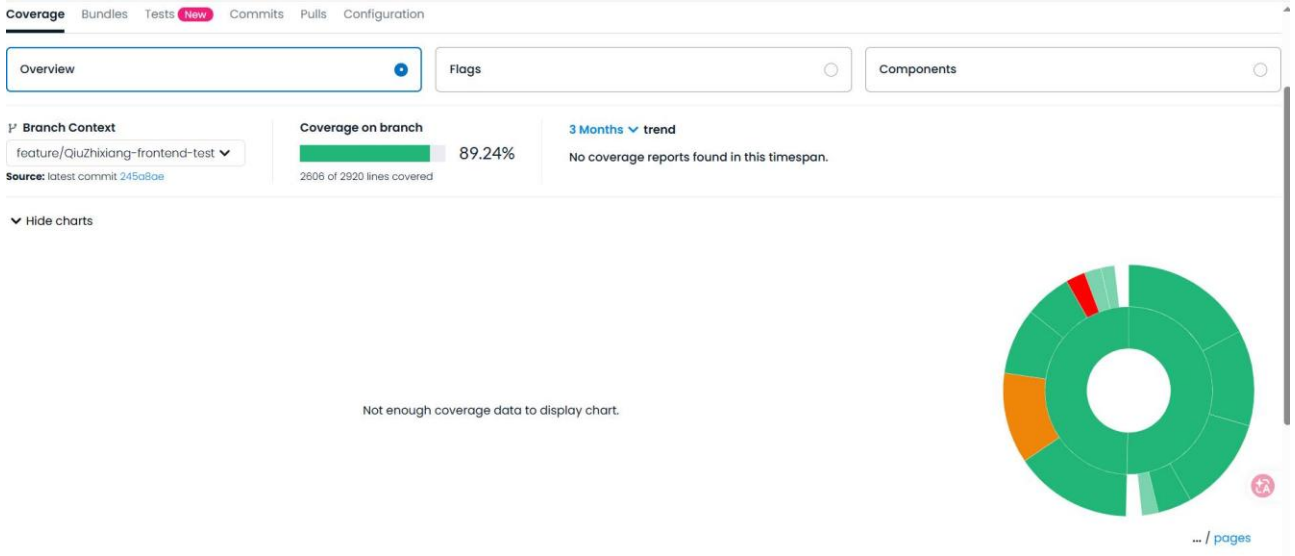


图 10-1 CI 中后端pytest 覆盖率报告截图与前端测试通过

十一、持续集成与持续交付（CI/CD） [何周屹]

11.1 CI/CD 方案

项目使用 GitHub Actions 作为 CI/CD 平台，.github/workflows 下有三条流水线：**ci.yml**（测试与 Lint）、**docker.yml**（镜像构建与扫描）、**security.yml**（密钥泄露扫描）。它们在 push 到 master/develop 及面向 master 的 PR 时触发，构成合并前的质量门禁。

11.2 自动化流水线

CI（ci.yml）分前后端两个 job 并行执行：

后端：在 Python 3.11 与 3.12 矩阵下，安装依赖 → ruff 代码检查 → 以 SQLite 内存库运行 pytest 并产出覆盖率 → 上传 Codecov。

```
9 jobs:
10   backend:
11     runs-on: ubuntu-latest
12     strategy:
13       matrix:
14         python-version: ["3.11", "3.12"]
15     env:
16       DATABASE_URL: "sqlite:///memory:"
17       USE MOCK DATA: "true"
18     steps:
19       - uses: actions/checkout@v4
20
21       - uses: actions/setup-python@v5
22         with:
23           python-version: ${ matrix.python-version }
24           cache: "pip"
25           cache-dependency-path: backend/requirements.txt
26
27       - run: python -m pip install --upgrade pip
28       - run: pip install -r backend/requirements.txt pytest coverage ruff
29
30       - run: ruff check backend/
31       - run: pytest backend/tests -c backend/pytest.ini --cov=app --cov-report=term-missing --cov-report=xml
32
33       - uses: codecov/codecov-action@v4
34         if: matrix.python-version == '3.12'
35         with:
36           files: ./coverage.xml
37           flags: backend
38           fail_ci_if_error: false
```

前端：Node 22 环境下 npm ci → npm run lint → npm test（带覆盖率）→ 上传 Codecov。

Docker（docker.yml）对 backend 与 frontend 两个镜像并行构建：PR 时本地构建并用 Trivy 扫描高危漏洞，push 时构建并推送到 GHCR，再对推送镜像做 Trivy 扫描，CRITICAL/HIGH 漏洞会让流水线失败。

```

40 frontend:
41   runs-on: ubuntu-latest
42   steps:
43     - uses: actions/checkout@v4
44
45     - uses: actions/setup-node@v4
46       with:
47         node-version: "22"
48         cache: "npm"
49         cache-dependency-path: frontend/package-lock.json
50
51     - run: npm ci --prefix frontend
52     - run: npm run lint --prefix frontend
53     - run: npm test --prefix frontend -- --coverage --ci
54
55     - uses: codecov/codecov-action@v4
56       with:
57         files: ./frontend/coverage/lcov.info
58         flags: frontend
59         fail_ci_if_error: false
60

```

Actions

New workflow

All workflows

CI

Dependabot Updates

Docker

Security Scan

Management

Caches

Attestations

Runners

Usage metrics

Performance metrics

61 workflow runs

Event Status Branch Actor

Merge pull request #89 from hzy2005/develop

CI #61: Commit b4e2c45 pushed by JoinMike333

master

Jun 17, 22:50 GMT+8

2m 17s

...

整合贡献文档

CI #60: Pull request #89 synchronize by JoinMike333

develop

Jun 17, 22:42 GMT+8

2m 12s

...

安全补丁升级

CI #59: Commit 53e16bf pushed by JoinMike333

develop

Jun 17, 22:42 GMT+8

2m 18s

...

整合贡献文档

CI #58: Pull request #89 reopened by JoinMike333

develop

Jun 17, 22:29 GMT+8

1m 53s

...

Upcoming change to GitHub App installation token format

GitHub App installation tokens will soon use a new stateless format (ghs_...) and may be longer (~520 characters). Apps with hardcoded length assumptions may break.

Validate your apps and workflows with the per-request override header detailed in this [GitHub Changelog](#).

CI

ci.yml

Filter workflow runs

Upcoming change to GitHub App installation token format

GitHub App installation tokens will soon use a new stateless format (ghs_...) and may be longer (~520 characters). Apps with hardcoded length assumptions may break.

Validate your apps and workflows with the per-request override header detailed in this [GitHub Changelog](#).

54 workflow runs

Event Status Branch Actor

This workflow has a workflow_dispatch event trigger.

Run workflow

Merge pull request #89 from hzy2005/develop

Docker #54: Commit b4e2c45 pushed by JoinMike333

master

Jun 17, 22:50 GMT+8

2m 7s

...

整合贡献文档

Docker #53: Pull request #89 synchronize by JoinMike333

develop

Jun 17, 22:42 GMT+8

1m 16s

...

安全补丁升级

Docker #52: Commit 53e16bf pushed by JoinMike333

develop

Jun 17, 22:42 GMT+8

1m 2s

...

Upcoming change to GitHub App installation token format

GitHub App installation tokens will soon use a new stateless format (ghs_...) and may be longer (~520 characters). Apps with hardcoded length assumptions may break.

Validate your apps and workflows with the per-request override header detailed in this [GitHub Changelog](#).

Docker

docker.yml

Filter workflow runs

Actions

New workflow

All workflows

CI

Dependabot Updates

Docker

Security Scan

Management

Caches

Attestations

Runners

Usage metrics

Performance metrics

Upcoming change to GitHub App installation token format

GitHub App installation tokens will soon use a new stateless format (ghs,...) and may be longer (~520 characters). Apps with hardcoded length assumptions may break. Validate your apps and workflows with the per-request override header detailed in this [GitHub Changelog](#).

Filter workflow runs

Security Scan

security.yml

51 workflow runs

Event Status Branch Actor

Merge pull request #89 from hzy2005/develop

Security Scan #51: Commit b4e2cd5 pushed by JoinMike333

master

Jun 17, 22:50 GMT+8

19s

整合贡献文档

Security Scan #50: Pull request #89 synchronize by JoinMike333

develop

Jun 17, 22:42 GMT+8

21s

安全补丁升级

Security Scan #49: Commit 53e16bf pushed by JoinMike333

develop

Jun 17, 22:42 GMT+8

17s

整合贡献文档

Security Scan #48: Pull request #89 reopened by JoinMike333

develop

Jun 17, 22:29 GMT+8

11s

整合贡献文档

Security Scan #47: Pull request #89 opened by JoinMike333

develop

Jun 17, 22:27 GMT+8

16s

图 11-1 GitHub Actions 流水线运行（CI / Docker / Security 三个 workflow 均通过）

11.3 分支保护与质量门禁

master 与 develop 配置分支保护：合并 PR 前要求 CI（测试、Lint、Docker 构建、安全扫描）全部通过，并要求至少一次代码审查；禁止直接推送。质量门禁确保进入主干的代码都经过自动化校验。

十二、系统部署 [邱志翔]

12.1 部署架构

采用前后端分离部署：后端 FastAPI 服务以 Docker 形式部署到 Render Web Service；前端为微信小程序，通过微信开发者工具上传体验版/正式版，接口指向线上后端 HTTPS 域名；数据库使用云 MySQL，后端通过 DATABASE_URL 连接。

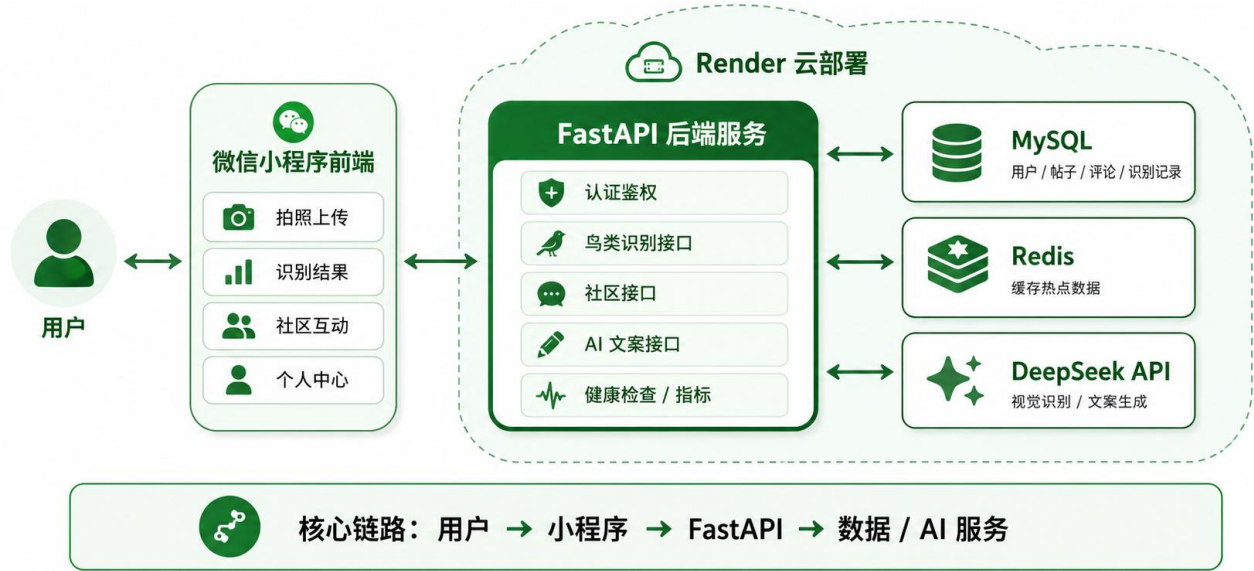


图 12-1 部署架构图（小程序 → Render 后端 → 云 MySQL）。

12.2 容器化

后端提供 backend/Dockerfile，容器启动时先执行 alembic upgrade head 完成数据库迁移，再用 Render 提供的 PORT 启动 Uvicorn。本地开发用 compose.yaml 一键拉起 backend、frontend、MySQL 8.4、Redis 7 四个服务，并为后端配置了基于 /health 的健康检查；生产用 compose.prod.yaml。

```
command: >
  sh -c "alembic upgrade head &&
  uvicorn app.main:app --host 0.0.0.0 --port 8000 --reload"
```

12.3 部署步骤

1. 登录 Render 控制台，New > Blueprint / Web Service，连接 GitHub 仓库 AviAI。
2. 选择部署分支（验证阶段用个人 feature 分支，联调用 develop，发布用 master）。
3. 使用仓库根目录的 render.yaml 创建服务（Docker，构建上下文 backend）。

- 在 Environment 页面配置环境变量（数据库、密钥、DeepSeek 等）。
- 点击 Deploy，等待构建与启动；开启 Auto Deploy 后推送即自动部署。
- 访问 `https://<域名>/api/v1/health` 验证，`database` 显示 `connected` 即部署成功。

12.4 环境配置

环境变量统一通过 `.env`（本地，已 `gitignore`）与平台环境变量（生产）管理，不写入仓库。`USE MOCK DATA` 用于区分开发与生产：开发可置 `true` 跳过真实 AI/DB 调用，生产置 `false`。

变量名	说明
<code>DATABASE_URL</code>	云 MySQL 连接地址
<code>SECRET_KEY</code>	JWT 签名密钥（长随机串）
<code>ACCESS_TOKEN_EXPIRE_MINUTES</code>	Token 有效期（分钟）
<code>USE MOCK DATA</code>	是否使用 Mock 数据（生产为 <code>false</code> ）
<code>DEEPSEEK_API_KEY</code> / <code>DEEPSEEK_BASE_URL</code> / <code>DEEPSEEK_MODEL</code>	DeepSeek AI 服务配置
<code>SENTRY_DSN</code> （可选）	错误追踪 DSN

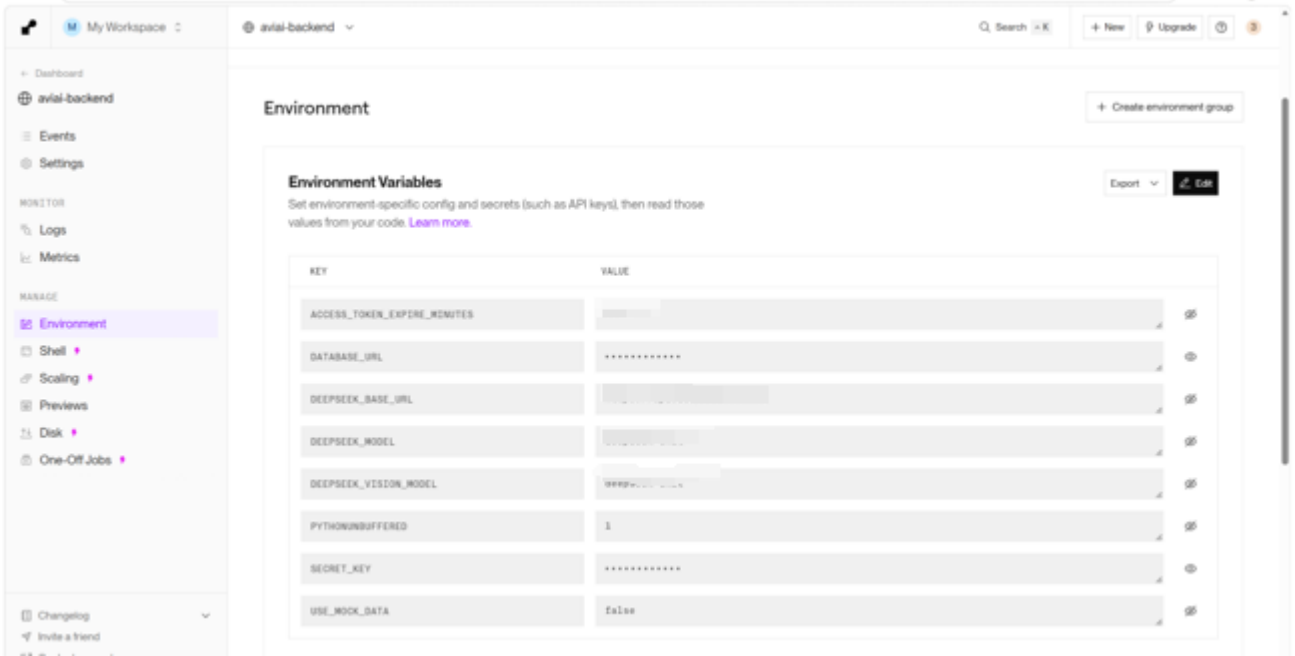


图 12-2 Render Environment 环境变量配置

十三、云服务应用 [邱志翔]

13.1 云平台选型

后端选择 Render 作为云平台。理由：支持 Docker Web Service，与项目已有的 backend/Dockerfile 天然契合；能连接 GitHub 仓库实现推送自动部署；自动提供 HTTPS 域名，便于在微信公众平台配置 request 合法域名。数据库使用云 MySQL，通过 DATABASE_URL 接入。

13.2 使用的云服务

服务类型	具体产品	用途
计算	Render Web Service (Docker)	运行 FastAPI 后端服务
数据库	云 MySQL	持久化用户、识别记录、社区数据
存储	容器卷 / uploads 目录	保存上传图片（可扩展为对象存储）
镜像仓库	GitHub Container Registry (GHCR)	存放 CI 构建的 Docker 镜像
监控/告警	UptimeRobot（可选）	外部可用性监控与告警

13.3 成本与资源配置

项目以课程验证为目标，主要使用免费套餐：Render 免费 Web Service 在空闲一段时间后会休眠，首次请求有冷启动延迟；云 MySQL 使用免费/最小规格实例。该配置满足演示与联调需求，但不适合高并发生产；正式上线时建议升级为常驻实例并接入对象存储

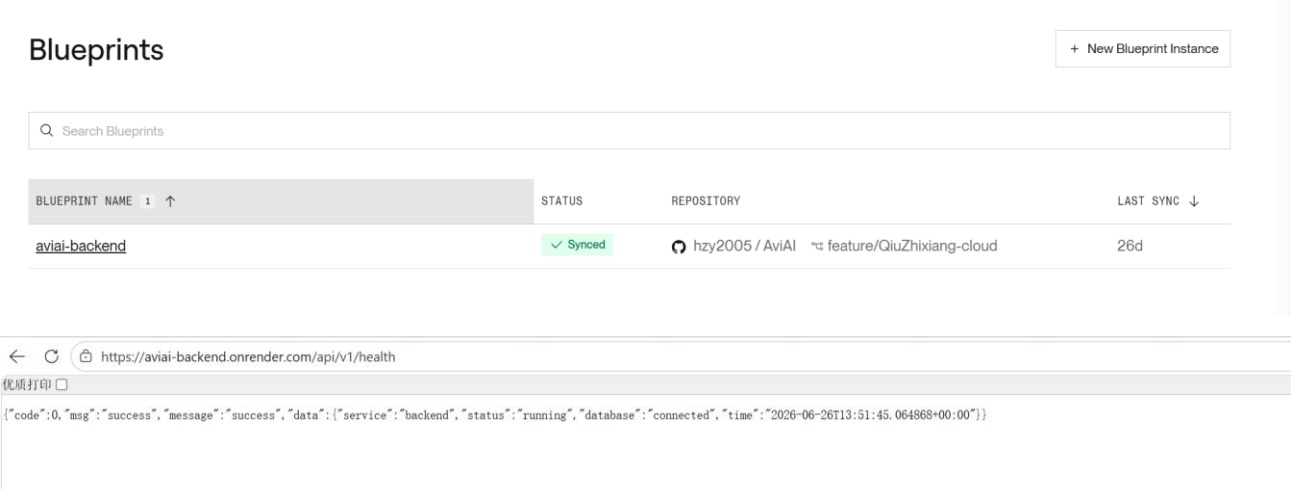


图 13-1 Render 服务部署成功与 /api/v1/health 返回

十四、可观测性与监控 [何周屹]

14.1 错误追踪

后端通过 `app/utils/sentry.py` 可选接入 Sentry：当配置了 `SENTRY_DSN` 时，服务启动自动初始化 Sentry FastAPI 集成，捕获未处理异常；DSN 为空时跳过初始化，不影响本地与普通部署。支持通过 `SENTRY_ENVIRONMENT`、`SENTRY_TRACES_SAMPLE_RATE` 配置环境与采样率。

14.2 日志管理

使用 `app/utils/logger.py` 输出 JSON 结构化日志，并接管 uvicorn 三个 logger 统一格式，避免 Render Logs 中混入默认文本日志。请求日志包含 `time`、`level`、`method`、`path`、`status_code`、`duration_ms`、`client` 等字段，便于按字段检索；日志默认输出到标准输出，在 Docker 日志与 Render Dashboard > Logs 中查看。

```
44 {
45     "time": "2026-05-31T09:00:00+00:00",
46     "level": "INFO",
47     "message": "request completed",
48     "module": "main",
49     "logger": "app.main",
50     "method": "GET",
51     "path": "/health",
52     "status_code": 200,
53     "duration_ms": 8.52,
54     "client": "127.0.0.1"
55 }
```

14.3 健康检查与可用性监控

提供两个健康检查端点：`/health` 返回简洁结构（`status/timestamp/version/database`），供 Render、Docker 与截图验收使用；`/api/v1/health` 返回统一响应结构供业务侧验证。Render 部署配置中健康检查路径设为 `/api/v1/health`；外部可用性用 UptimeRobot 每 5 分钟探测 `/health`，状态异常时邮件告警。

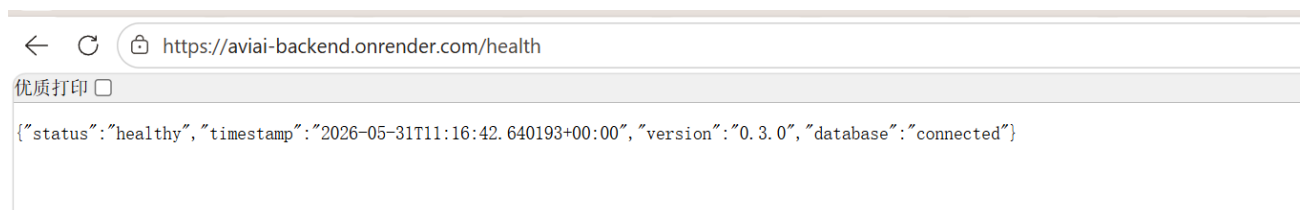


图 14-1 UptimeRobot 监控详情（状态 Up、检查间隔、告警联系人）。

14.4 指标监控

`/api/v1/metrics` 返回进程内基础指标（请求数、错误数、错误率、平均响应时间、活跃请求数、各状态码计数）；`/metrics` 额外提供 Prometheus 文本格式，便于后续接入 Prometheus / Grafana 抓取。当前指标来自进程内存统计，适合单实例部署，多实例场景建议接入独立指标存储。

```
value: |
# HELP aviai_requests_total Total number of processed HTTP requests.
# TYPE aviai_requests_total counter
aviai_requests_total 155
# HELP aviai_average_response_ms Average response time in milliseconds.
# TYPE aviai_average_response_ms gauge
aviai_average_response_ms 614.72
aviai_status_codes_total{status_code="200"} 155
```



图 14-2 `/api/v1/metrics` 接口返回

十五、性能优化 [何周屹、邱志翔]

15.1 性能基线报告

性能观测分前后端两条线。后端以 /api/v1/metrics 的平均响应时间与状态码分布作为基线；前端以微信开发者工具的 Performance 面板观察启动与渲染。关键指标关注：

- ①后端接口平均响应时间与 P99（结合指标接口与压测）。
- ②小程序冷启动时间与列表滚动帧率（移动端）。
- ③识别接口端到端耗时（含模型推理）。

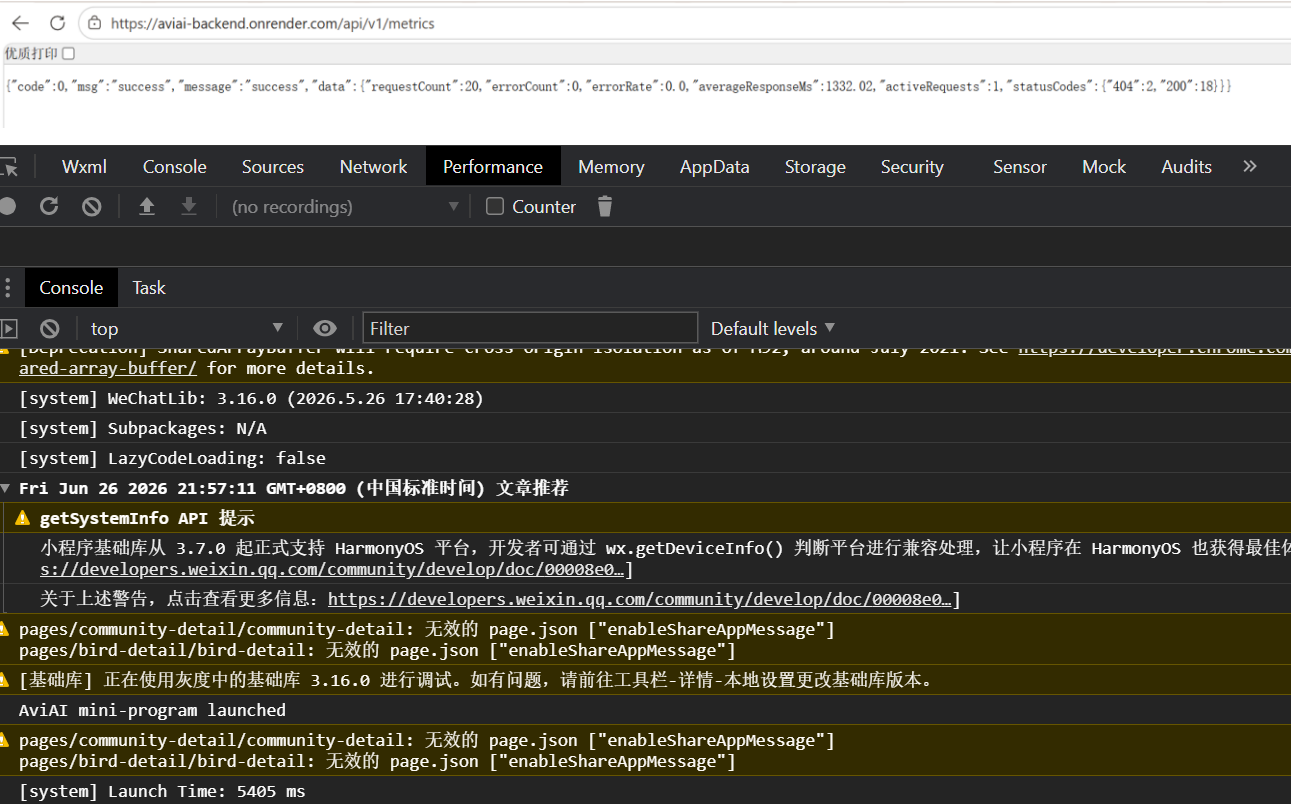


图 15-1 性能基线（后端 /api/v1/metrics 指标以及微信开发者工具 Performance 面板）

15.2 已完成的优化项

优化项	优化前	优化后	说明
识别模型加载	每次请求加载 ResNet18	进程内单例懒加载	首次后推理延迟大幅降低
列表查询	全量返回	分页 + 索引	响应数据量与耗时下降
时间线读取	直查 MySQL	Redis 缓存热点	热点读延迟降低

.

前端长列表	一次性渲染	触底分页加载	首屏更快、滚动更顺
上传图片	原图直传	压缩后上传	上传耗时与流量下降

注：表中 Before/After 的精确数值建议在压测/真机实测后补全，可借助 `/api/v1/metrics` 与开发者工具 Network 面板取数。

十六、功能展示 [何周屹、邱志翔]

16.1 系统演示

围绕核心链路演示：注册登录 → 拍照识别鸟类并查看结果 → 浏览百科/历史 → 发布社区动态（含 AI 文案）→ 点赞评论互动。建议将同一模块的截图拼接成组图以节省篇幅。

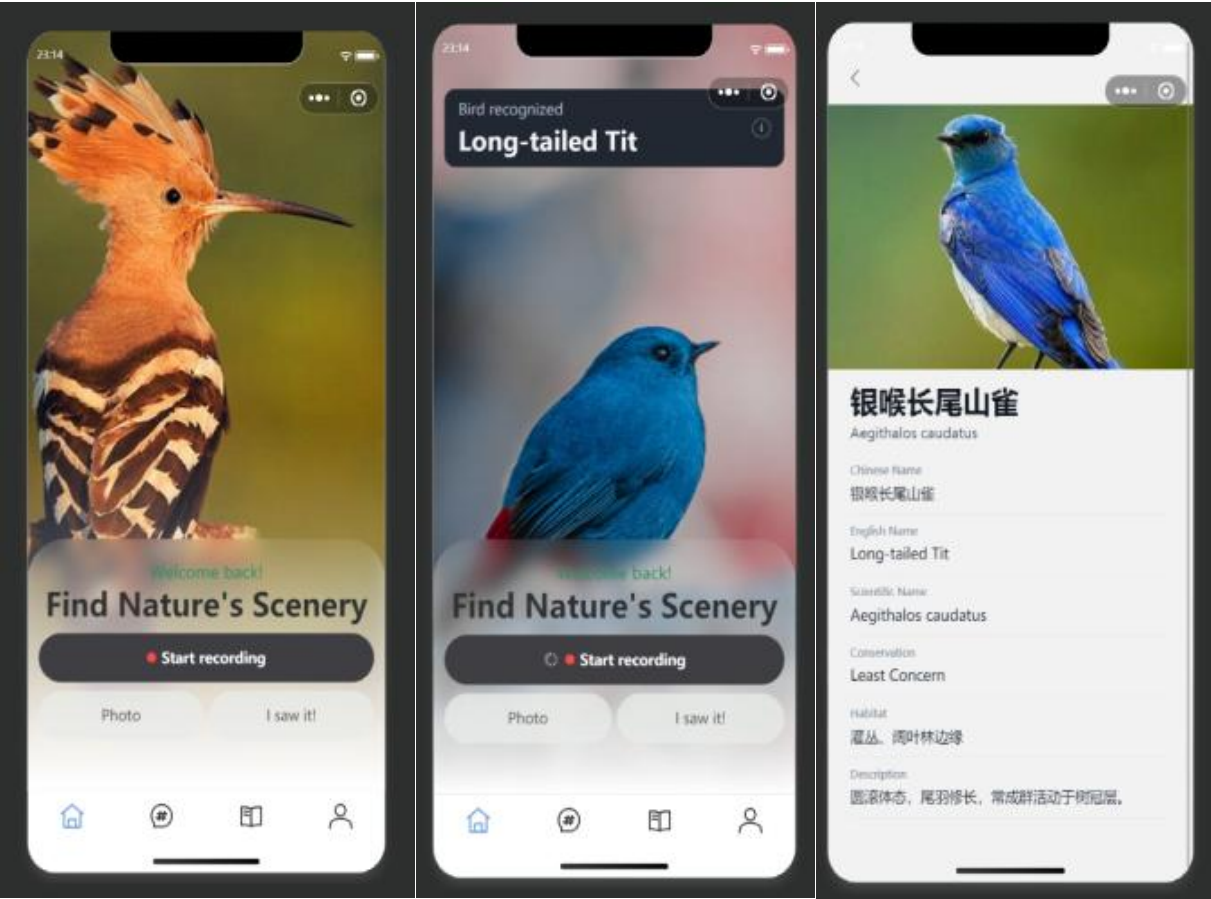


图 16-1 识别功能演示组图

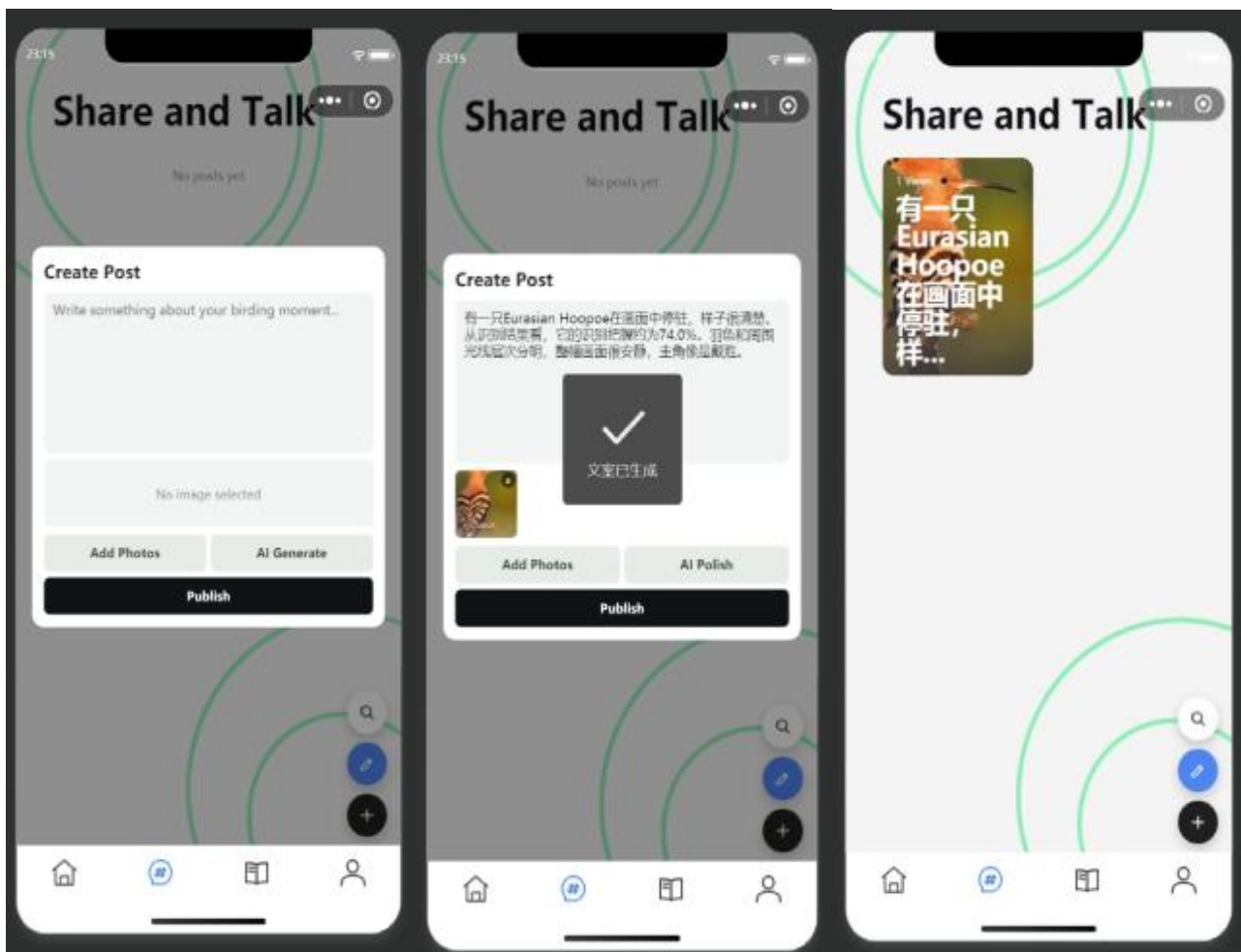




图 16-2 社区功能演示组图

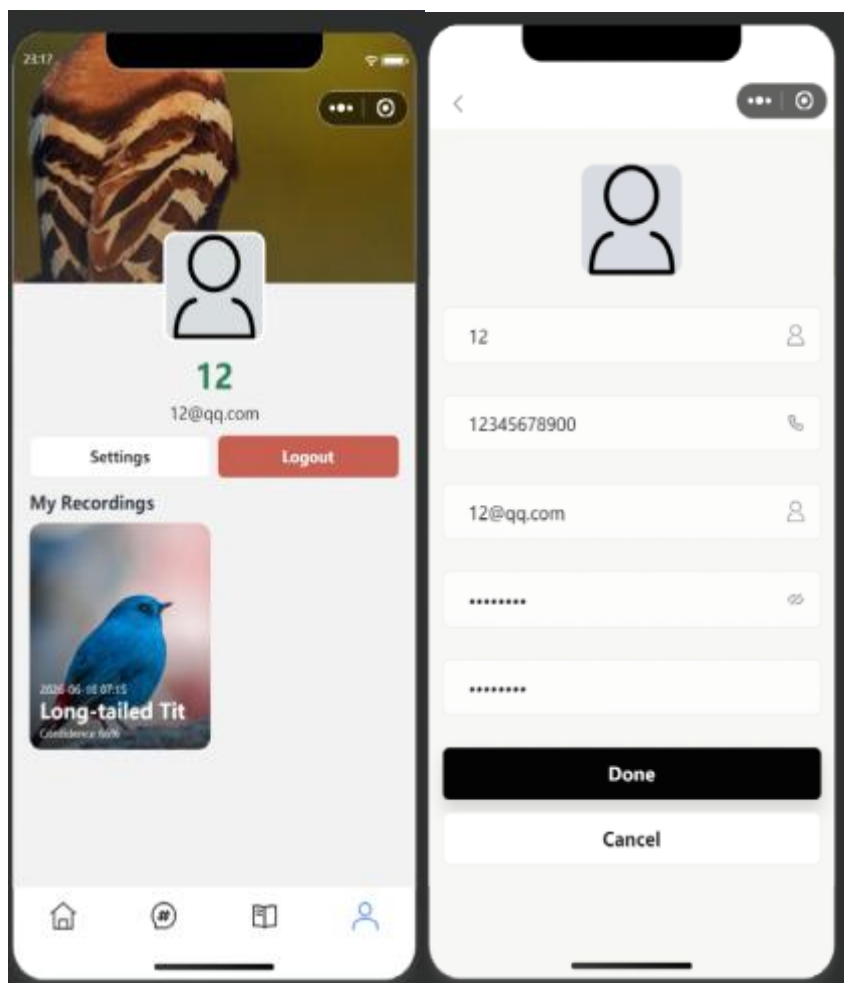


图16-3 账号与个人中心演示组图

16.2 性能测试结果

在正常负载下，后端健康检查与读接口响应稳定，`/api/v1/metrics` 显示错误率为 0、平均响应时间处于可接受范围。建议附一次连续访问后的指标快照作为佐证。

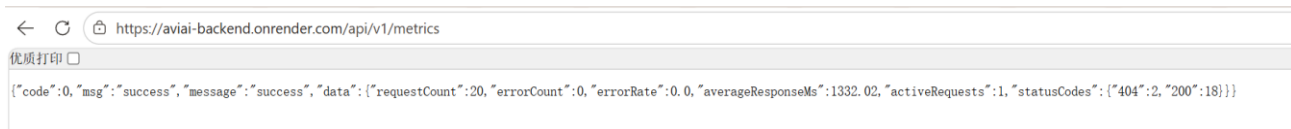


图16-4 正常负载下 `/api/v1/metrics` 指标快照

十七、总结与展望 [何周屹、邱志翔]

17.1 项目总结

AviAI 项目围绕鸟类识别与观鸟社区这一主题，完成了从需求分析、界面设计、前后端开发到部署测试的完整实践。系统已经实现账号注册登录、鸟类图片识别、识别历史记录、鸟类百科、社区发布与互动、AI 文案辅助、个人中心等核心功能，前端小程序与 FastAPI 后端能够端到端联通，具备真实部署和演示的条件。

在工程化方面，项目接入了自动化测试、CI/CD、Docker 容器化、云端部署、安全扫描和基础监控。后端测试数量约 104 个，行覆盖率约 84%，能够覆盖认证、识别、社区、监控等主要业务路径。整体来看，项目基本达成了预期的功能目标，也在稳定性、安全性和可维护性方面做了相应支撑。

17.2 技术收获

通过本项目的开发，我们对 FastAPI 后端分层架构有了更具体的理解，也实际使用了 Pydantic 参数校验、SQLAlchemy ORM、Alembic 数据库迁移等后端工程技术。认证模块中，我们实现了基于 Token 的登录鉴权与退出失效机制，对用户认证、会话管理和接口权限控制有了更深入的认识。

前端部分加深了我们对原生微信小程序开发方式的掌握，包括页面组织、状态管理、图片上传、页面跳转、缓存使用和统一请求封装。项目中还处理了本地环境、局域网环境和线上环境之间的切换问题，这让我们更直观地理解了前后端联调中的环境配置与接口契约管理。

在项目交付层面，我们实践了从本地开发到 CI/CD、Docker、云部署、监控和安全检查的完整流程。AI 在项目中既作为开发辅助工具，也作为实际产品能力参与到社区文案生成中。这个过程也提醒我们，AI 输出可以提升效率，但仍需要结合业务场景、测试结果和人工判断进行核对。

17.3 问题与反思

项目开发过程中也暴露出一些不足。监控相关测试曾出现不稳定情况，主要原因是请求计数和并发状态存在时序影响，测试断言写得过于严格。这说明自动化测试不仅要覆盖功能，还要避免对执行顺序和瞬时状态产生过强依赖。

文档维护方面也存在偏差。早期部分文档中保留了 Node/Express 等初始设想，而最终实现已经调整为 FastAPI。这个问题反映出项目迭代过程中，文档需要随着代码同步更新，否

则容易影响后续验收、维护和团队沟通。后续应当以实际代码和接口契约为准，及时清理过期内容。

部署与存储方面，目前图片主要采用本地文件存储，适合课程项目和单实例部署，但不利于多实例扩展和长期稳定运行。免费云平台也存在冷启动和资源限制，这些问题让我们意识到，真实项目中需要更早考虑对象存储、服务扩展、运行成本和稳定性之间的平衡。

17.4 未来展望

后续如果继续完善 AviAI，可以在识别能力上引入更专业的鸟类细分类模型，提升物种识别粒度和准确率，让识别结果更贴近真实观鸟场景。图片存储也可以迁移到对象存储，减少本地文件依赖，为后端多实例部署和长期运行打下基础。

功能层面，系统可以继续扩展附近鸟类分布、基于地理位置的观测推荐、用户关注、消息通知和社区互动增强等能力，让平台从“识别工具”进一步发展为“观鸟记录与交流平台”。测试方面也可以引入端到端自动化测试，覆盖小程序关键操作路径，进一步提升系统稳定性和回归验证效率。

AI 使用声明

本文档及项目中以下部分由 AI 辅助生成，并经团队人工审核与修改：

章节 / 环节	AI 工具	使用方式	人工修改情况
第一章 项目介绍	ChatGPT	生成初稿框架	补充真实数据与背景，核对功能范围
第五章 API 设计	GitHub Copilot	生成接口描述与示例	逐条核对与代码 / api.yaml 一致性
第九章 安全设计	ChatGPT	OWASP 视角安全审查	核实漏洞与修复，去除误报
代码开发	GitHub Copilot	补全样板代码与测试脚手架	核对逻辑、补充边界用例

未在上表中列出的章节均由团队成员独立撰写。AI 生成内容均经过人工核对，确保与实际代码一致。

第三方库与开源引用

本项目使用的主要第三方库及开源组件如下，均通过包管理器（pip / npm）引入，未直接复制源码：

库 / 框架	版本	用途	来源
FastAPI	0.136.1	后端 Web 框架	https://fastapi.tiangolo.com
Uvicorn	0.32.0	ASGI 服务器	https://www.uvicorn.org
SQLAlchemy	2.0.49	ORM 数据库访问	https://www.sqlalchemy.org
Alembic	1.14.0	数据库迁移	https://alembic.sqlalchemy.org
PyMySQL	1.1.1	MySQL 驱动	https://pymysql.readthedocs.io
bcrypt	>=4.2.0	密码哈希	https://pypi.org/project/bcrypt
PyTorch / torchvision	>=2.3.0	ResNet18 图像分类	https://pytorch.org
Pillow	12.2.0	图像处理	https://python-pillow.org
sentry-sdk	2.19.2	错误追踪	https://sentry.io

pytest / httpx

—

测试框架与 HTTP
客户端

<https://pytest.org>
